Векторные представления текста

Static и contextual embeddings,
transfer learning, semantic search и RAG

Алан-Барсаг Газзаев

Содержание второй лекции

Static и document embeddings

Сначала рассматривается связь между corpus, контекстом слова и его representation. Word2Vec, GloVe и fastText сравниваются по источнику обучающего сигнала и способу работы со словами. Затем обсуждаются representations предложений и документов: TF-IDF baseline, pooling word embeddings и doc2vec. Для каждого подхода важно понимать, какая информация сохраняется и что теряется.

Contextual embeddings и применение

ELMo и BERT вводят representation конкретного употребления слова. Transfer learning позволяет использовать pretrained model и адаптировать её к downstream task. Sentence embeddings связывают эти идеи с semantic search и RAG. Лекция объясняет понятия и выбор компонентов; код, получение embeddings и сравнение classifiers относятся к семинару.

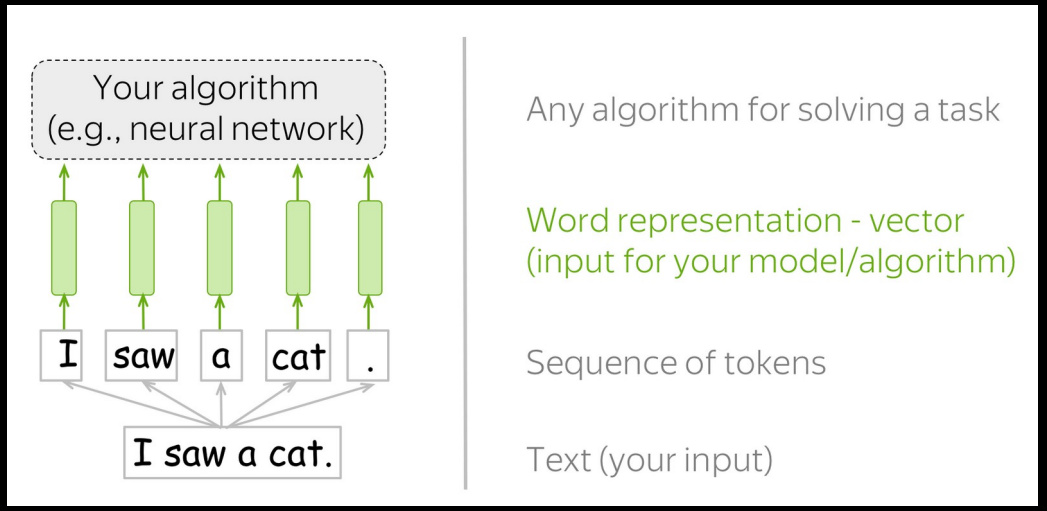
Уровни text representation

Объект, которому соответствует вектор, необходимо определить до выбора модели. Одинаковое слово embedding может обозначать разные уровни анализа.

Объект	Что описывает vector	Пример применения
Token / word	Элемент или его употребление	Sequence labeling
Sentence / passage	Предложение или фрагмент	Semantic search
Document	Документ целиком	Classification, clustering

Word embedding не становится sentence embedding автоматически. Для перехода между уровнями требуется pooling или отдельная модель.

Text representations в NLP pipeline



Исходный текст сначала превращается в последовательность tokens. Модель получает численные representations этих элементов, а затем строит ответ на прикладную задачу. На схеме word representations являются входом для следующего алгоритма. Смысл каждой координаты обычно не задан заранее: полезная информация распределена по вектору и проявляется в сравнении с другими representations.

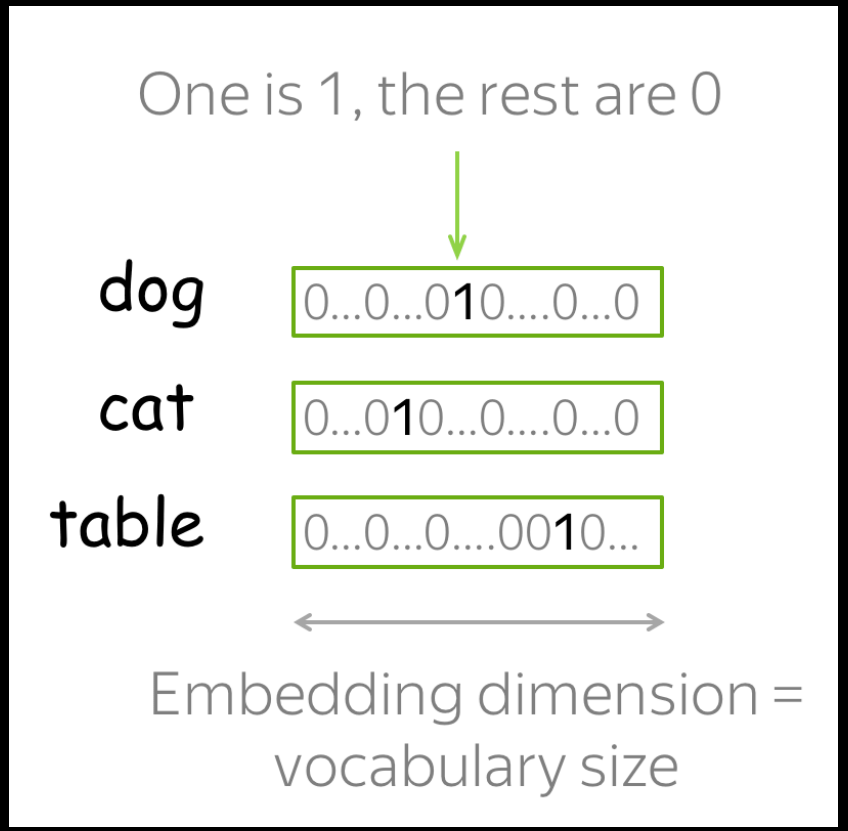
Sparse и dense representations

Различие относится к организации численного описания. Оно само по себе не определяет качество решения.

Свойство	Sparse representation	Dense representation
Координаты	Часто связаны с vocabulary	Обычно обучаются совместно
Нулевые значения	Преобладают у коротких текстов	Обычно не преобладают
Пример	Counts, TF-IDF	Word и sentence embeddings

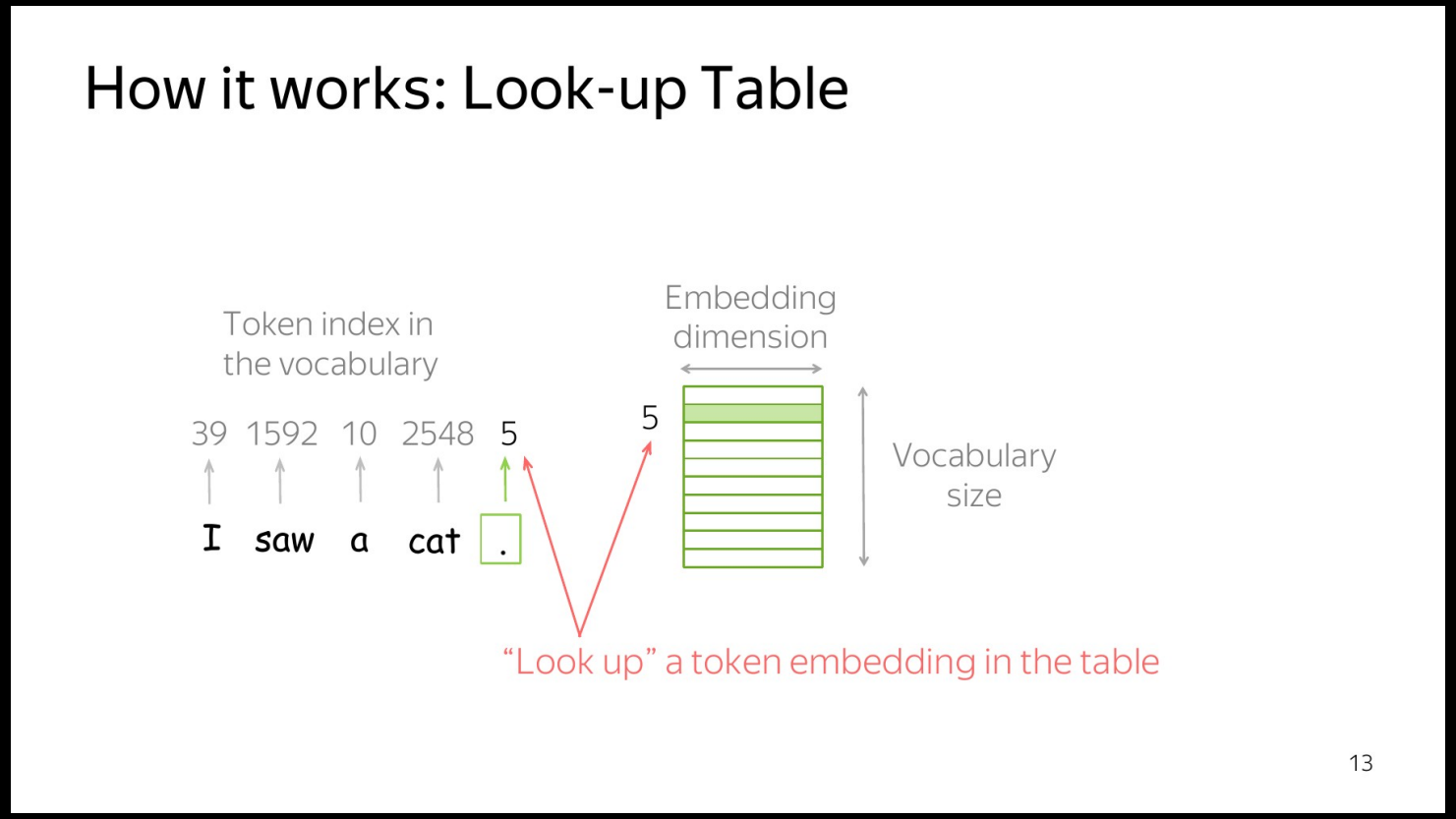
Sparse features удобны для точных lexical signals. Dense embeddings позволяют учитывать статистические связи между разными формулировками, но тоже зависят от данных и задачи.

One-hot representation



В one-hot vector одна координата равна единице, остальные — нулю. Координата выбирается по позиции слова в vocabulary. Размер representation совпадает с размером vocabulary; разные слова получают разные независимые координаты. Такой способ сохраняет идентичность элемента, но сам по себе не сообщает, что cat ближе по смыслу к dog, чем к table. Для смысловых отношений необходим дополнительный обучающий сигнал.

Embedding lookup table



Token ID выбирает строку embedding matrix. Vocabulary и embeddings должны сохранять одно и то же соответствие.

Distributional hypothesis

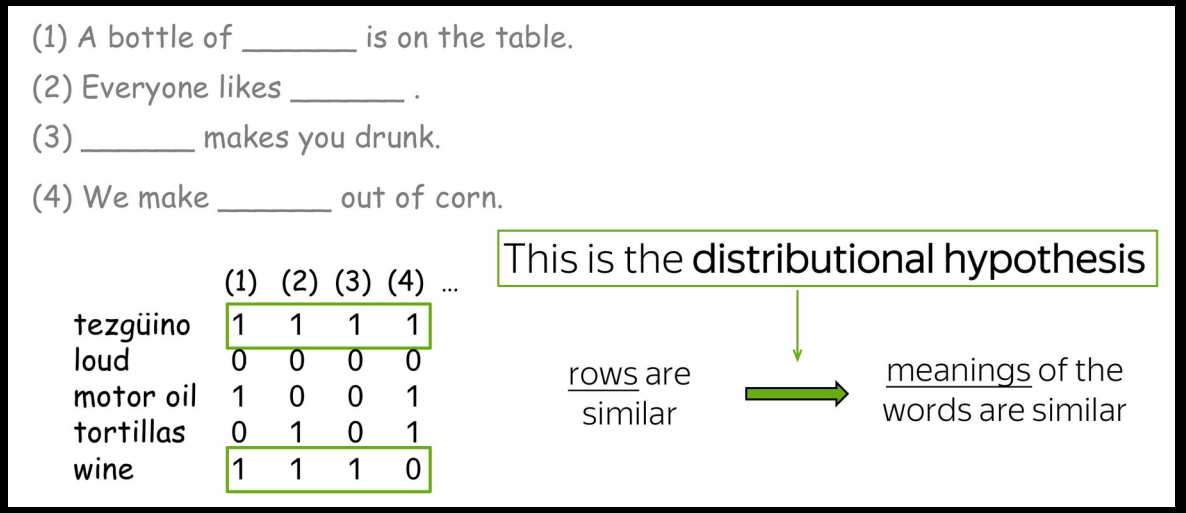
Контексты как наблюдаемые данные

Distributional hypothesis связывает смысловую близость со сходством контекстов употребления. Если два слова встречаются в похожих окружениях, по corpus можно обнаружить между ними статистическую связь. Это даёт обучающий сигнал без ручной разметки значения каждого слова. При этом контекст определяется выбранным методом: соседние слова, предложение или документ.

Границы предположения

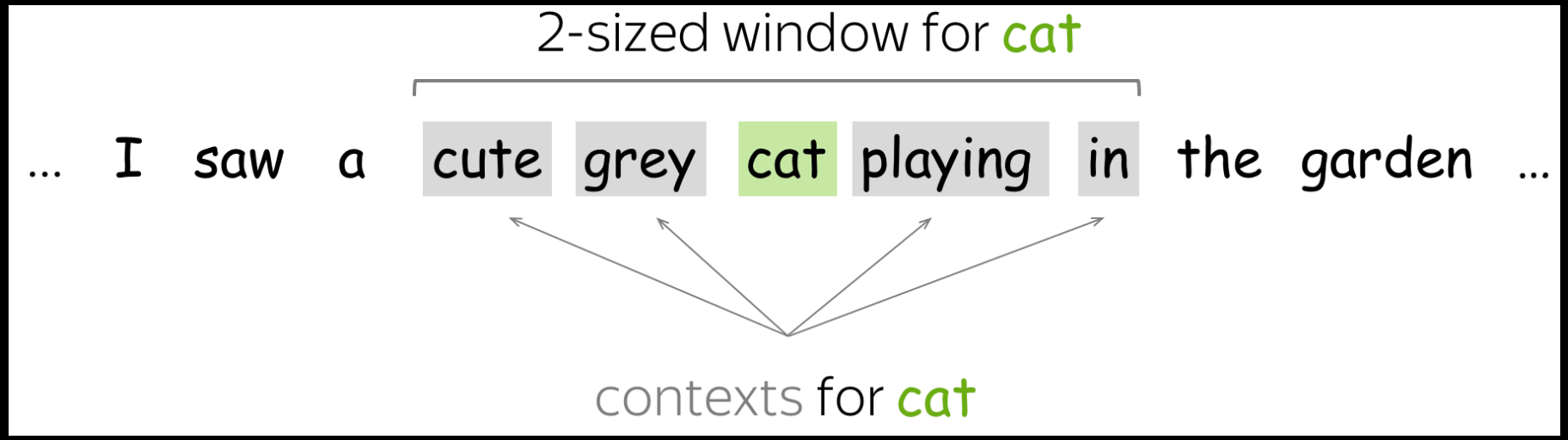
Похожее окружение не означает полную взаимозаменяемость. Antonyms, слова одной темы и названия конкурирующих объектов тоже могут встречаться в сходных контекстах. Representation отражает закономерности corpus и цель обучения. Поэтому близость vectors интерпретируют относительно задачи, а не как универсальное определение одинакового значения.

Значение слова через контексты



В примере курса значение незнакомого слова tezgüino постепенно становится понятнее по окружающим выражениям. Каждое употребление ограничивает возможную интерпретацию. Аналогично corpus даёт модели множество наблюдений о слове, даже когда определение не предоставлено. Один случай может быть неоднозначен; устойчивые закономерности обнаруживаются при сопоставлении разных контекстов.

Context window



Context window определяет, какие соседние слова считаются окружением текущего слова. Окно перемещается по corpus и создаёт наблюдения для разных позиций. Размер window влияет на характер информации: ближайшие слова часто связаны с локальной конструкцией, более широкое окружение может отражать тему. Конкретный результат зависит также от corpus, weighting и preprocessing.

Выбор контекста

Локальный и широкий контекст

Для слова bank ближайшие элементы river и credit дают разные сигналы, хотя сама запись остаётся одинаковой. Контекст можно ограничить фиксированным числом tokens, предложением или более крупной единицей. Если границы документов игнорируются, в обучающие наблюдения могут попасть искусственные соседства, которых нет в реальном языке.

Параметры representation

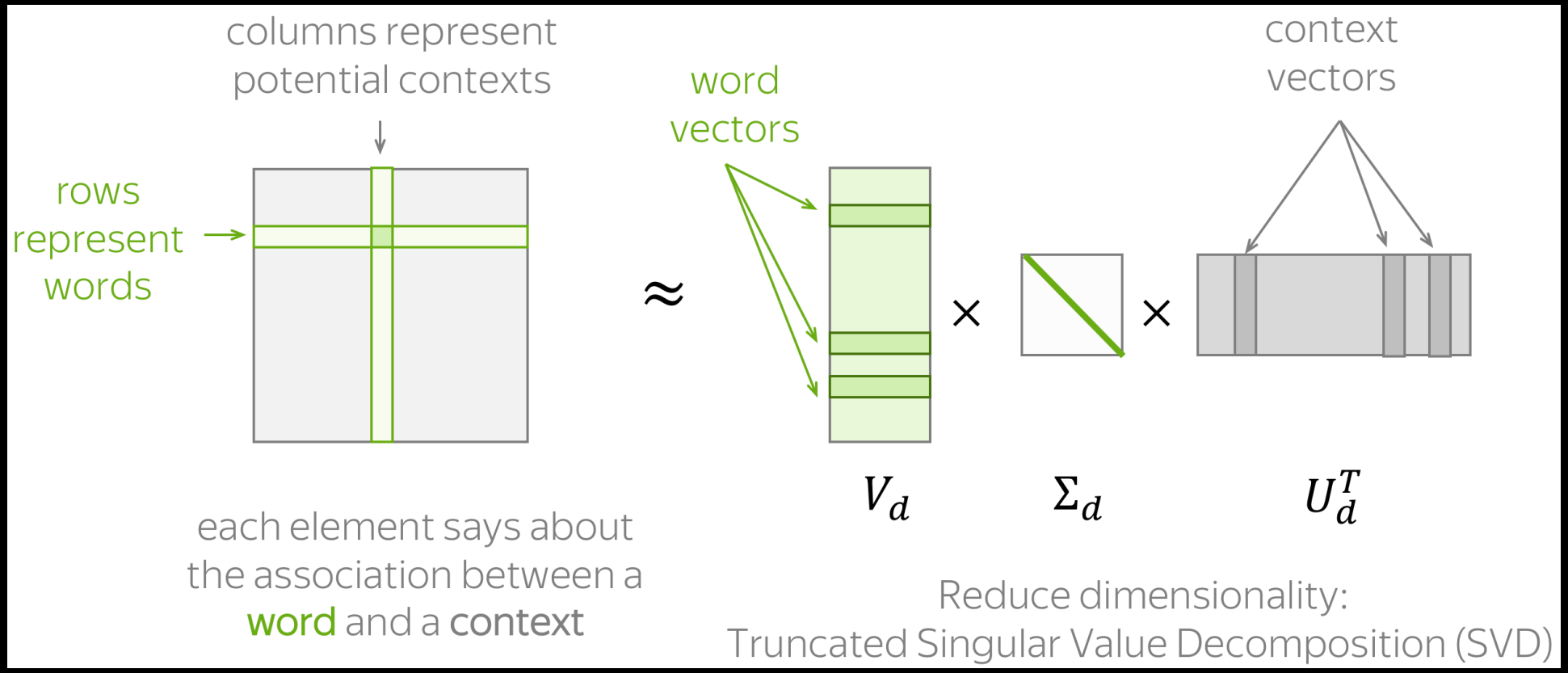
Учитывается не только размер window, но и направление, расстояние до слова и правила исключения элементов. Узкий контекст и широкий контекст отвечают разным исследовательским вопросам. Выбор должен проверяться по полезности representations в downstream task. Универсального размера window для всех языков и коллекций не существует.

Word–context matrix



Строка word–context matrix описывает, с какими context words встречалось выбранное слово. Счётчики собираются по corpus. Сходство строк показывает сходство наблюдаемых окружений; редкие слова имеют меньше надёжных наблюдений. Матрица может быть очень большой и sparse. Поэтому используют weighting и уменьшение размерности, чтобы получить более компактное representation.

Count-based representations



Статистика word–context преобразуется в компактные representations. Выбор контекстов определяет исходную информацию.

Dimensionality reduction

Компактное описание

Разные context words могут появляться вместе и нести близкую информацию. Dimensionality reduction описывает часть этой структуры меньшим числом координат. Например, LSA строит низкоразмерное приближение term–document matrix: исходные признаки объединяются в общие направления, отражающие совместное употребление слов.

Выбор embedding dimension

Число координат задаёт ёмкость representation и влияет на хранение и вычисления. Слишком маленький vector может потерять полезные различия; увеличение dimension не гарантирует рост качества при ограниченных данных. Координаты нового пространства обычно не соответствуют отдельным словам и не получают однозначных человеческих названий.

TF-IDF как baseline

Representation документа

В TF-IDF документ задаётся весами terms из vocabulary. Координаты имеют понятную связь с текстом, а различия формулировок учитываются только через выбранные features и preprocessing. Такой baseline особенно полезен, когда в задаче важны точные слова, номера, редкие названия или устойчивые сочетания. Получение features не требует обучения большой neural model.

Сравнение с embeddings

TF-IDF и dense embeddings сравнивают на одинаковом split и с одинаковыми labels. Dense representation может сопоставлять тексты без общих слов, но иногда теряет точные lexical distinctions. Поэтому переход к embeddings является гипотезой об улучшении, которую нужно проверить. Baseline позволяет измерить полезность дополнительной сложности.

Word vectors и document vectors

Сначала определяют, какой объект описывает vector и как получены его координаты. Одинаковая численная размерность не означает одинаковую интерпретацию.

Representation	Один объект	Содержание координат
Word–context	Слово	Его окружения в corpus
TF-IDF document	Документ	Вес каждого term
Sentence embedding	Предложение	Обучаемые общие признаки

Нельзя сравнивать word vector и document vector только потому, что у них одинаковая длина. Они должны находиться в согласованном пространстве и быть предназначены для такого сравнения.

Embedding space

Отдельный vector и пространство

Embedding dimension задаёт длину representation. Один vector интерпретируется через отношения с другими vectors, полученными совместимым способом. Близкие точки могут соответствовать похожим текстам или словам, если это поддерживается обучающей целью. Сам номер координаты обычно не имеет фиксированного значения вроде «тональность» или «тема».

Совместимость координат

Два независимо обученных пространства нельзя напрямую смешивать. Даже если оба используют 300 координат, направление одной оси не обязано совпадать. Замена encoder или его weights обычно требует повторного вычисления всех representations в index. Совместимость определяется моделью и настройками, а не только размерностью.

Cosine similarity

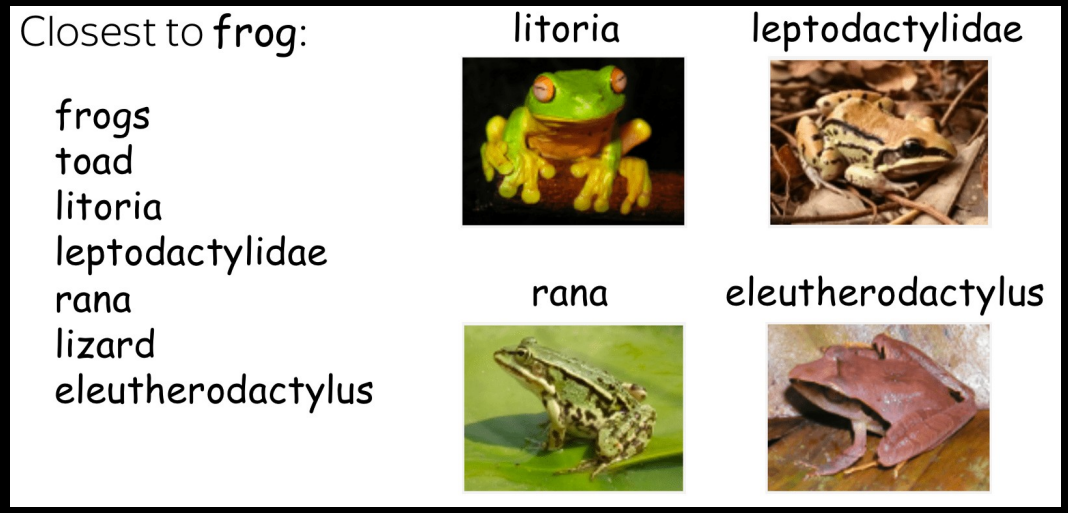
Сравнение направлений

Cosine similarity сравнивает направления vectors. При совпадающем направлении score максимален; противоположное направление даёт отрицательное значение. Длина vector при таком сравнении исключается через normalization. Геометрический смысл показателя не означает, что он автоматически измеряет semantic similarity: это зависит от того, как получены embeddings.

Интерпретация score

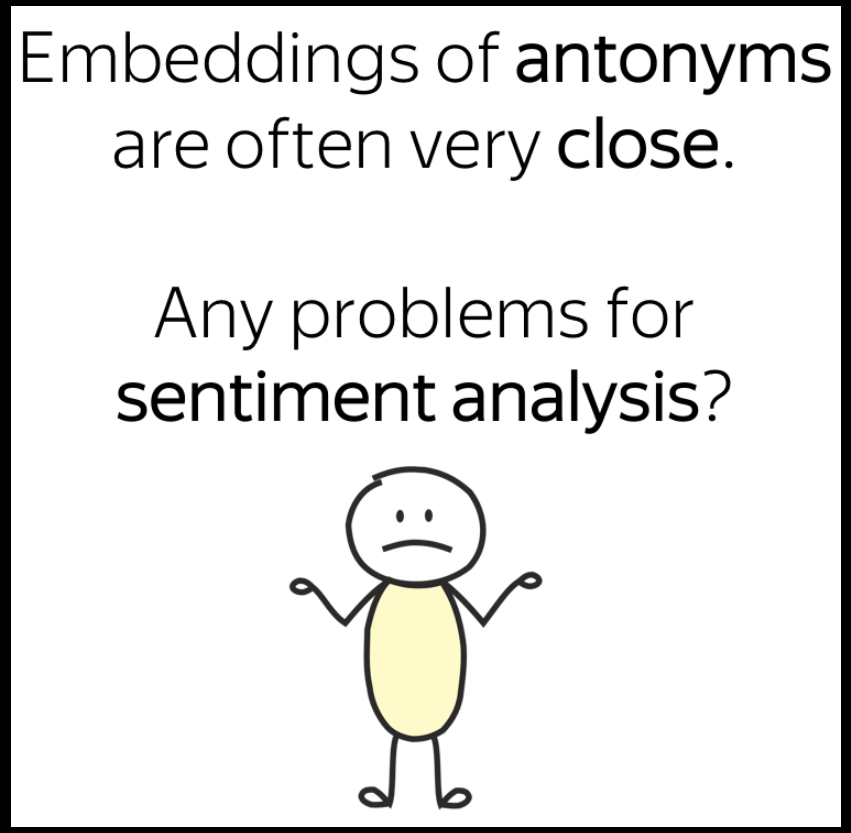
Cosine score не является вероятностью релевантности и не задаёт универсальный порог. Распределение значений меняется между моделями, языками и задачами. Для поиска важен порядок candidates; для решения «похожи или нет» порог выбирают на размеченных примерах. Для zero vector cosine similarity не определена без дополнительного соглашения.

Nearest neighbors



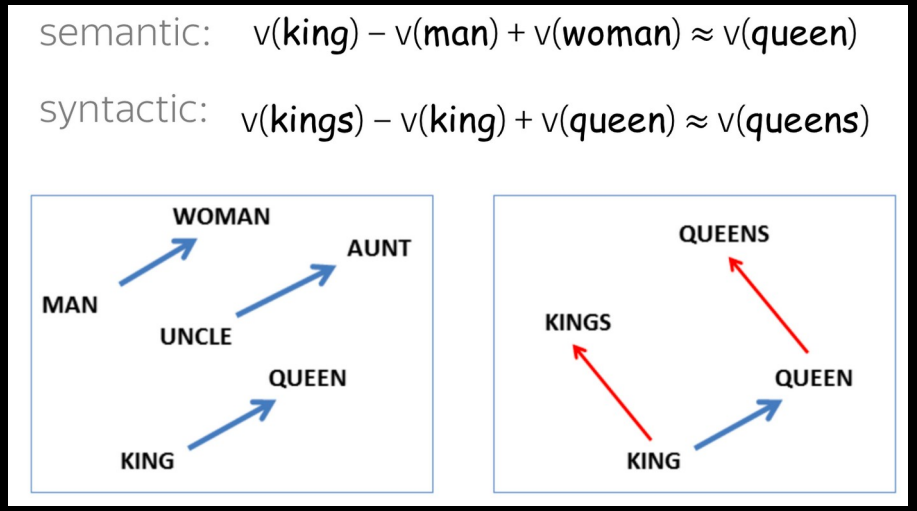
Nearest neighbors — элементы с наиболее близкими vectors по выбранной similarity measure. Пример из курса показывает соседей слова frog: среди них есть родственные формы и другие названия животных. Такой просмотр помогает понять структуру пространства и обнаружить проблемы vocabulary. Он не заменяет evaluation на прикладной задаче: убедительные отдельные примеры не характеризуют всю модель.

Similarity и relatedness



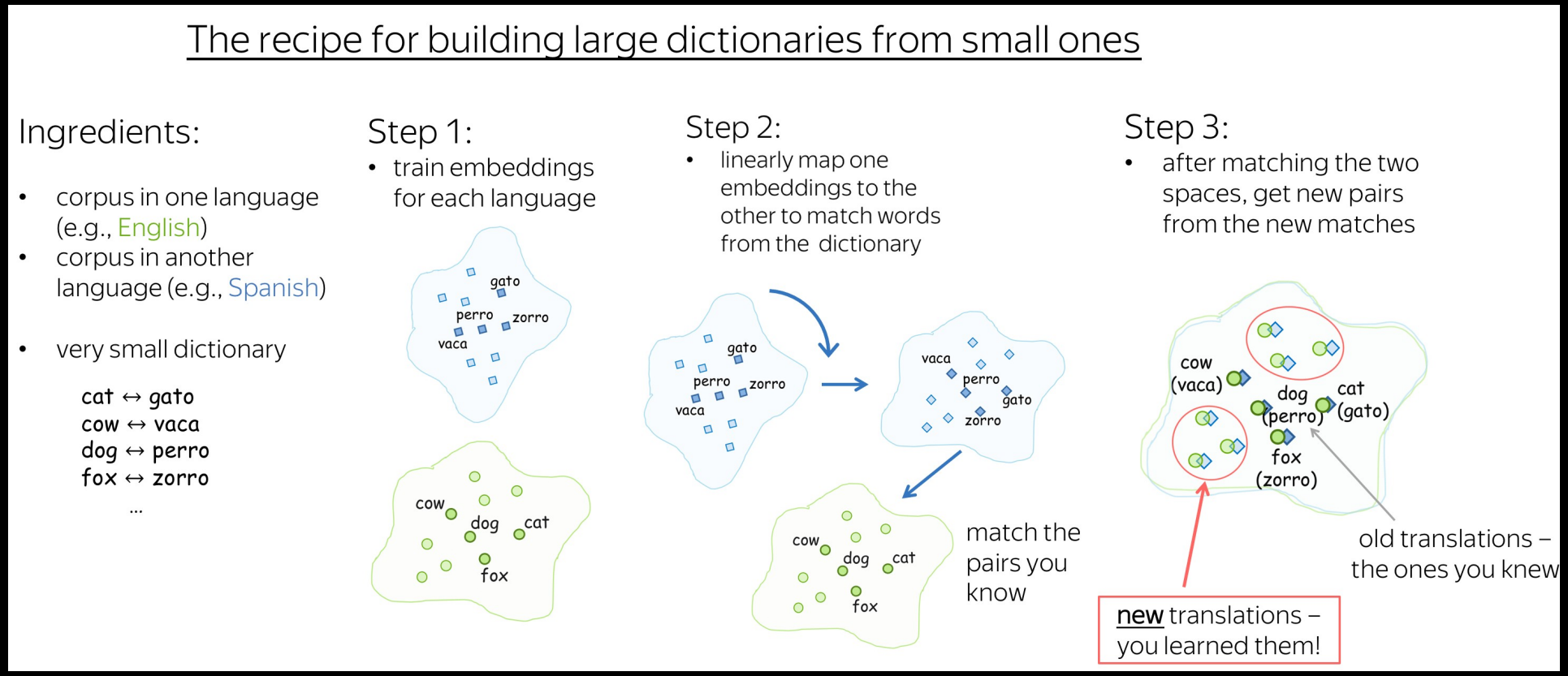
Слова good и bad могут иметь близкие контексты: оба описывают качество товара, фильма или обслуживания. Поэтому distributional representations способны сближать antonyms. Для sentiment analysis это не означает одинаковую class label. Classifier должен научиться использовать нужные различия, а при необходимости representation дополнительно адаптируют к задаче. Геометрическая близость и одинаковый ответ модели — разные свойства.

Word analogies



Word analogy проверяет, сохраняются ли некоторые отношения между парами слов в геометрии пространства. Известные примеры сопоставляют роли или формы слов. На схеме показана иллюстрация такого отношения, а не универсальное правило рассуждения. Результат зависит от vocabulary, corpus и способа поиска ответа; удачная analogy не доказывает понимания всех свойств связанных понятий.

Embedding spaces разных языков



Сходные отношения между словами не означают совпадения координат. Для сравнения языков требуется согласование пространств.

Ограничения embedding space

Corpus и устойчивость

Representations зависят от источников, времени и языка corpus. Для частых слов доступны многочисленные контексты; для редких слов оценки менее устойчивы. В пространстве могут отражаться стереотипы и перекосы данных. Оценивать embeddings только по нескольким красивым примерам опасно: нужны систематические проверки на выбранной задаче.

Визуализация и выводы

Проекция в две координаты удобна для обзора, но искажает часть расстояний и отношений. Cluster на картинке не является доказательством отдельной смысловой категории. Анализ должен учитывать исходную similarity measure и настройки проекции. Особенно осторожно интерпретируют большие расстояния и направление осей в t-SNE-подобных визуализациях.

Word2Vec

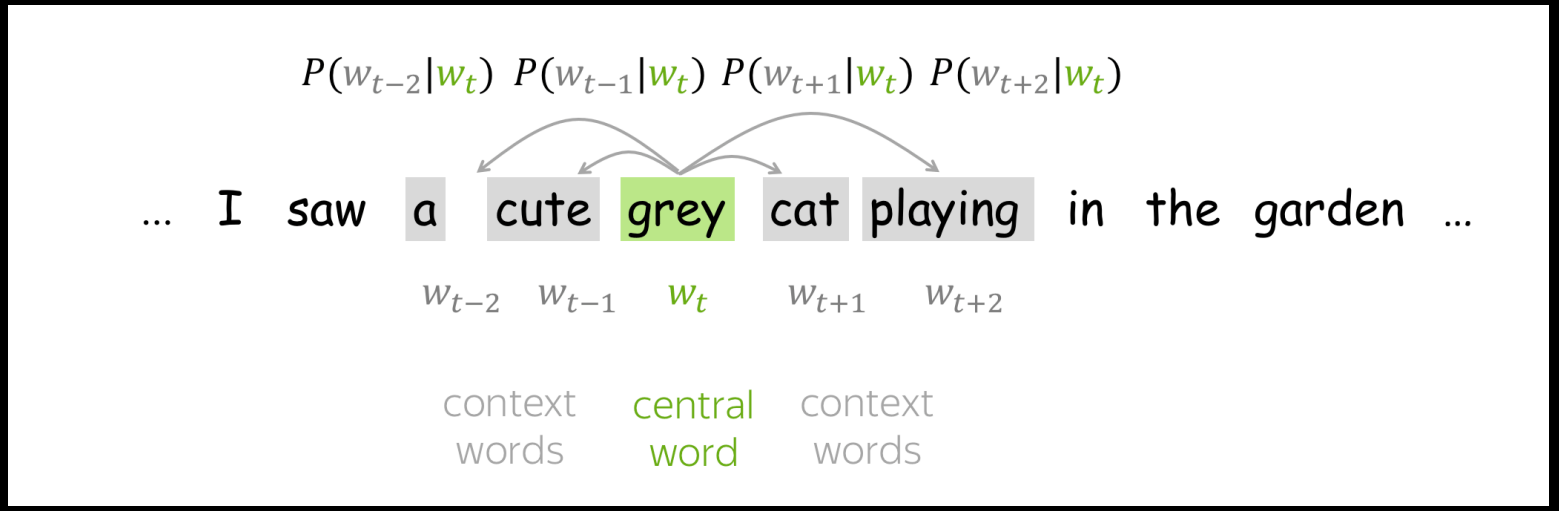
Prediction-based representation

Word2Vec обучает word vectors через задачу предсказания слов по контексту или контекста по слову. Размеченные class labels для каждого документа не требуются: обучающие пары получают из последовательности tokens. Цель обучения заставляет vectors отражать регулярности окружения. Итоговые representations затем применяют в другой задаче.

Что остаётся после обучения

Для каждого слова из vocabulary сохраняется static vector. В обычном Word2Vec разные употребления одной словоформы получают один и тот же vector. Это делает lookup быстрым, но объединяет разные значения слова. Обучающая задача Word2Vec и конечная задача, например classification отзывов, обычно не совпадают.

Skip-gram



Skip-gram использует central word для предсказания context words в выбранном window. Одно положение окна даёт несколько обучающих пар. По мере перемещения окна разные слова становятся центральными и получают сигналы от своих окружений. На схеме изображён один фрагмент этого процесса; модель не получает словарного определения слова, а учится по повторяющимся контекстам.

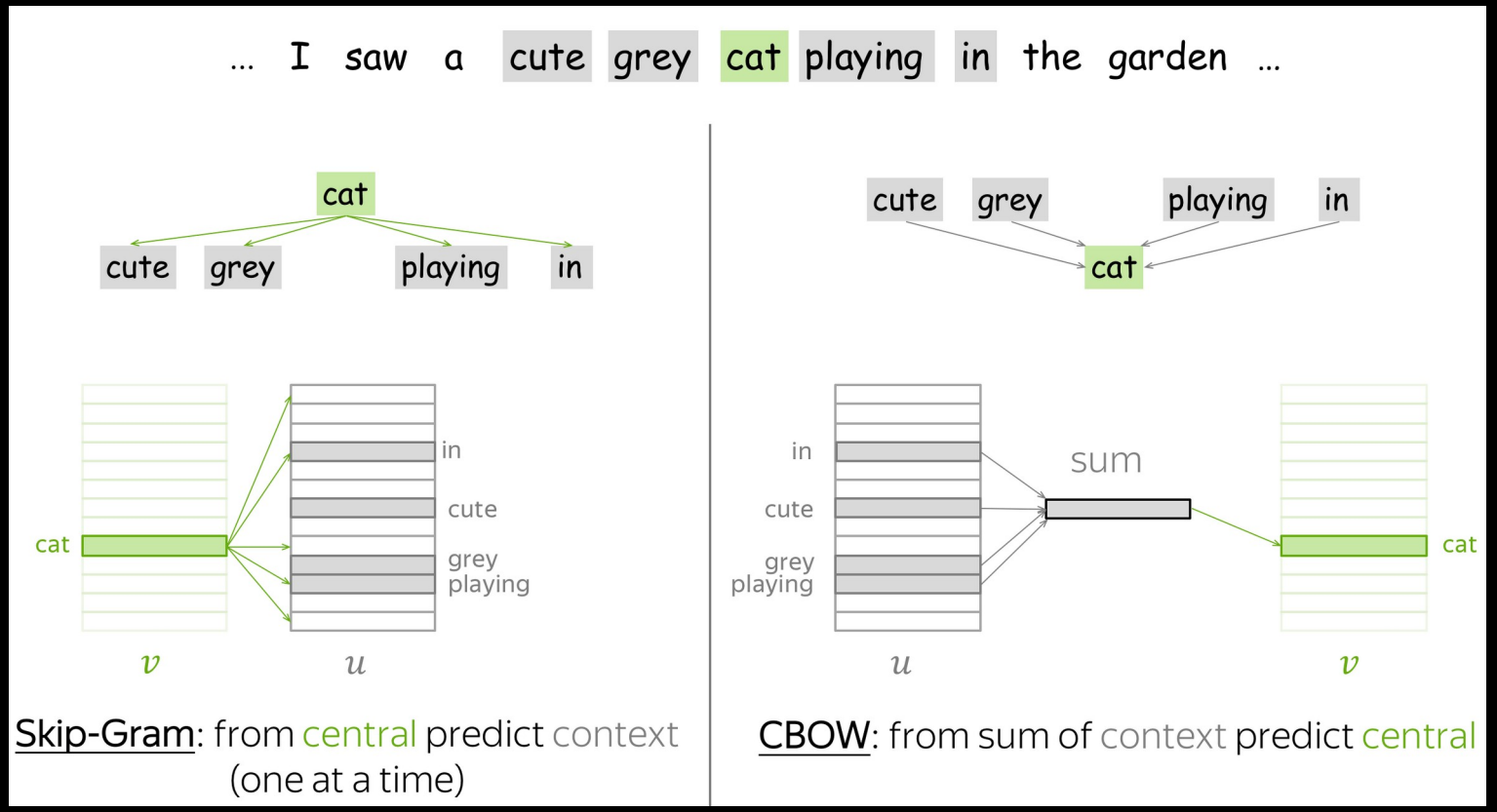
Обучающие пары для Skip-gram

Учебный пример: «студент читает полезную книгу». Central word — «читает»; window содержит по одному соседу с каждой стороны.

Central word	Context word	Наблюдение
читает	студент	Сосед слева
читает	полезную	Сосед справа

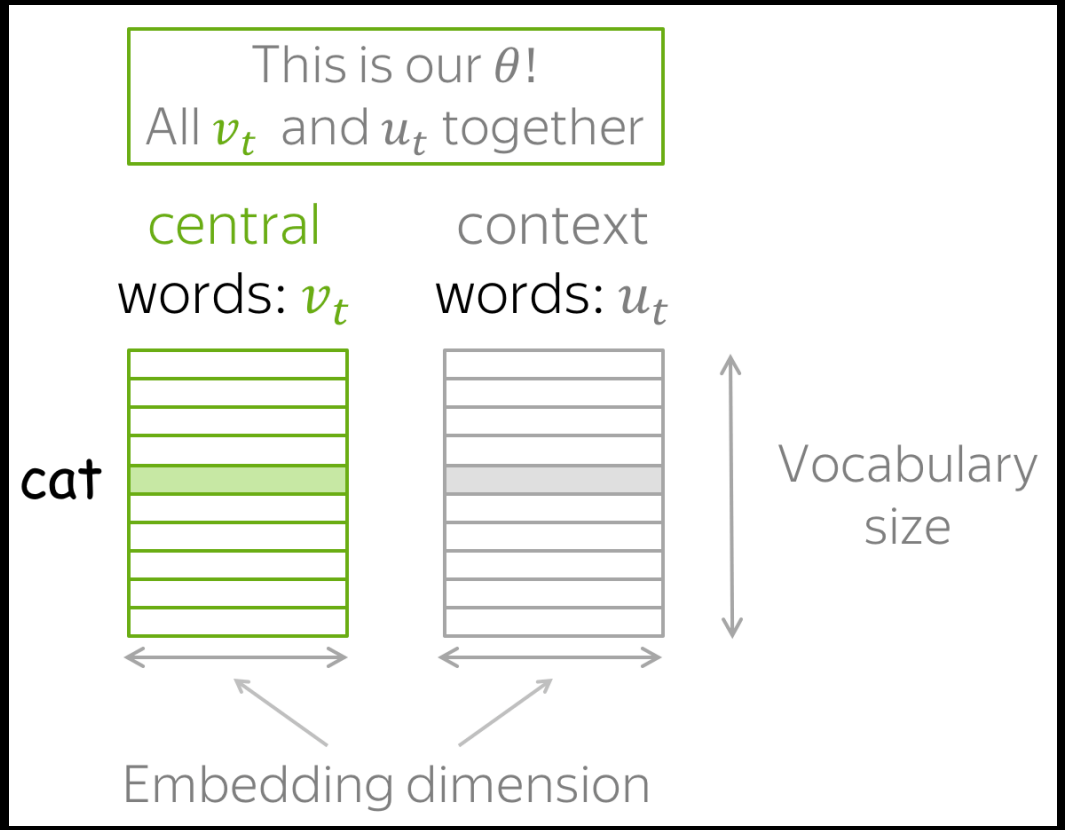
При следующем положении window изменятся central word и пары. Здесь показана только организация данных, без расчёта probabilities и обновления weights.

CBOW и Skip-gram



Skip-gram предсказывает context words по central word; CBOW предсказывает central word по его окружению.

Word и context embeddings



В стандартном обучении Word2Vec различаются роли слова как входа и как context target. Для этих ролей используются два набора обучаемых vectors. На схеме курса показаны две соответствующие таблицы. При выгрузке pretrained embeddings часто сохраняют только один набор. Поэтому формат готового файла может содержать меньше информации, чем полная обучаемая модель.

Negative sampling

Вместо полного перебора vocabulary модель сопоставляет наблюдаемую пару с небольшим набором sampled alternatives. Ниже — условный пример организации данных.

Central word	Context candidate	Источник пары
cat	cute	Наблюдение в window
cat	invoice	Sampled negative
cat	station	Sampled negative

Negative означает выбор процедурой sampling, а не доказанную невозможность контекста. Число и распределение sampled words влияют на обучающий сигнал.

Corpus для Word2Vec

Подготовка наблюдений

Corpus определяет доступные значения слов и типичные контексты. Сохраняются границы документов и согласованные правила tokenization. Нормализация регистра, обработка punctuation и исключение редких слов меняют vocabulary и обучающие пары. Важно заранее решить, должны ли имена, числа, сокращения и специальные tokens сохраняться как самостоятельные элементы.

Данные и задача применения

Большой corpus общего назначения не обязательно лучше небольшой специализированной коллекции для узкой терминологии. Но маленькая коллекция может не дать достаточно наблюдений для устойчивых word vectors. Поэтому часто начинают с pretrained embeddings и проверяют их coverage и качество. Обучение с нуля имеет смысл, когда данные и цель такого эксперимента определены.

Параметры Word2Vec

Параметры меняют доступные наблюдения, ёмкость representation и процедуру обучения. Их выбирают под данные и downstream evaluation.

Параметр	Что меняется	Что проверять
Window size	Доступный контекст	Локальные связи и темы
Embedding dimension	Размер vector	Качество и затраты памяти
Min count	Состав vocabulary	Coverage редких слов
Sampling settings	Обучающий сигнал	Устойчивость результата

Большее значение параметра не означает автоматически лучшее representation. Сравнение требует одинаковой процедуры оценки.

Context window и обучаемые отношения

Состав обучающих пар

Увеличение window добавляет более удалённые context words. Это может усилить тематические связи и одновременно ослабить относительную роль ближайших соседей. Направление и расстояние также можно учитывать отдельно. Изменение window фактически меняет данные, которые модель считает подтверждением связи между словами.

Проверка полезности

Близость в embedding space зависит от этой процедуры. Для одних задач полезнее общая тема, для других — точная роль слова в локальной конструкции. Поэтому размер window не выбирают по одному примеру nearest neighbors. Проверяют несколько классов слов и downstream task, фиксируя остальные настройки и источник corpus.

Pretrained word embeddings

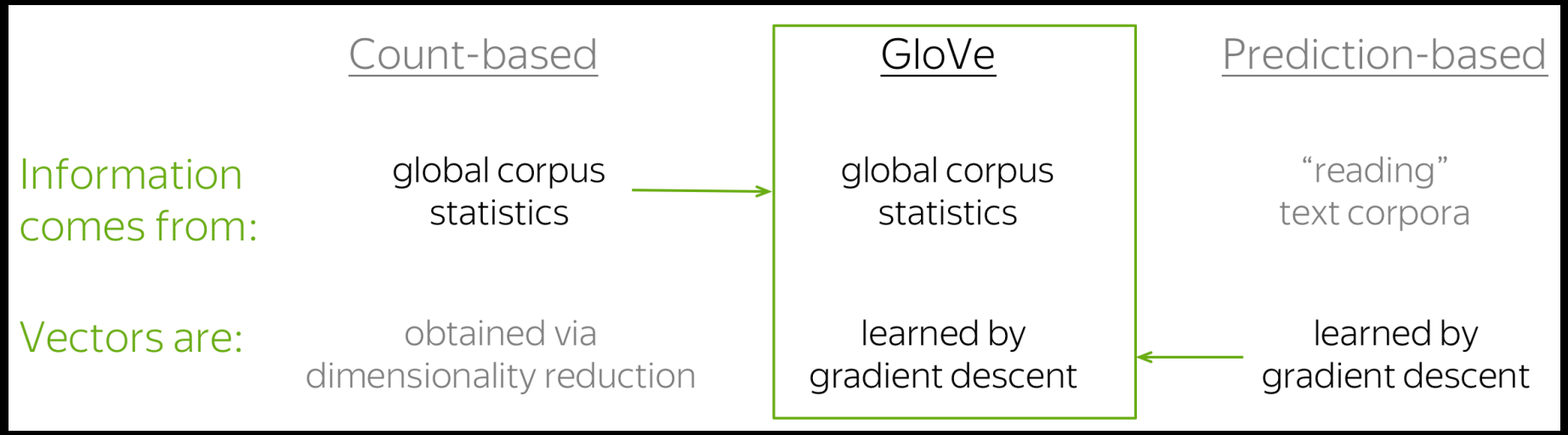
Повторное использование vectors

Готовые word embeddings позволяют начать работу без собственного pretraining. Перед применением проверяют язык, vocabulary, регистр, tokenization и embedding dimension. В Gensim объект KeyedVectors хранит соответствие между ключом и vector и поддерживает поиск похожих элементов. Он не обязательно содержит всё состояние, нужное для продолжения обучения исходной модели.

Coverage и воспроизводимость

Coverage показывает, какая часть нужных слов представлена в выбранном наборе. Важна не только общая доля, но и пропуски среди имён, терминов и редких классов. Сохраняют источник и версию файла, правила обработки OOV и способ построения document representation. Замена набора vectors меняет пространство и требует повторной оценки.

GloVe



GloVe использует агрегированные word–word co-occurrence statistics. Сначала по corpus собирают совместные появления, затем обучают компактные word vectors, отражающие эти связи. Название означает Global Vectors for Word Representation. В результате получается static representation слова. В отличие от простого хранения строки counts, GloVe обучает более короткий vector по статистике всей коллекции.

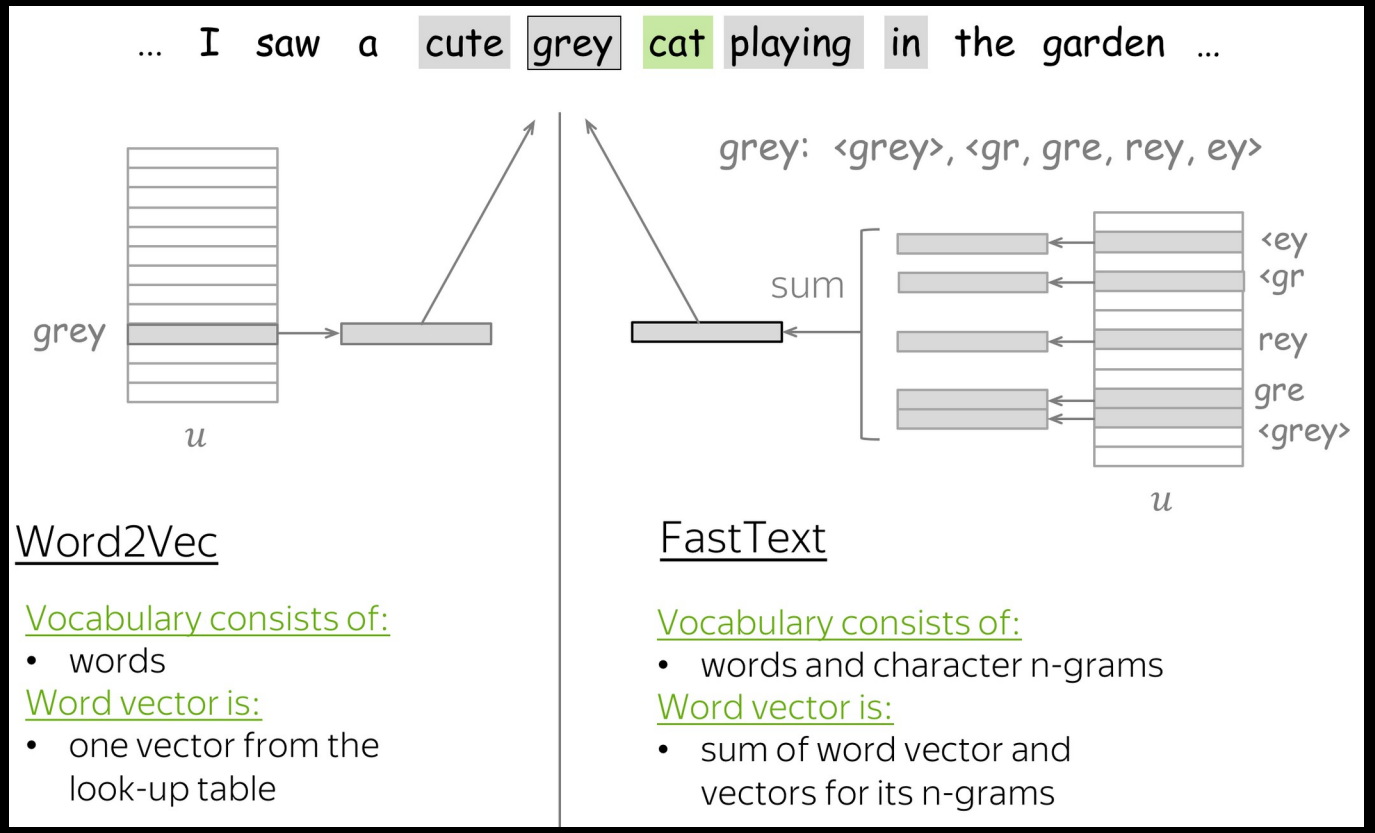
Word2Vec и GloVe

Оба подхода связывают слово с его контекстами и дают static word vectors. Различается организация обучающего сигнала.

Свойство	Word2Vec	GloVe
Исходные наблюдения	Word–context pairs	Co-occurrence statistics
Основная идея	Предсказывать окружение	Согласовать vectors со статистикой
Результат	Static word vectors	Static word vectors

Выбор нельзя обосновать только названием метода. Существенны corpus, vocabulary, dimension, реализация и качество в целевой задаче.

fastText: subword information



В fastText word vector использует character n-grams. Это даёт общий обучающий сигнал для слов с похожей записью.

fastText и OOV

Representation нового слова

Полная fastText model может построить vector для слова, отсутствовавшего в vocabulary целых слов, используя параметры его character n-grams. Это полезно для новых словоформ и некоторых опечаток. Возможность вычислить vector не означает, что модель знает значение слова: результат зависит от знакомых фрагментов и обучающего corpus.

Формат модели

Текстовый файл с готовыми vectors для перечисленных слов обычно не сохраняет все subword parameters. Поэтому простая выгрузка word vectors и полная fastText model дают разные возможности для OOV. При выборе файла проверяют, требуется ли только lookup известных слов или вычисление representations для новых записей.

Subword information: примеры

Character n-grams связывают слова по записи. Это полезный сигнал, но совпадение написания не является доказательством одинакового значения.

Случай	Что может помочь	Ограничение
«книга» / «книги»	Общие фрагменты словоформ	Нужные различия могут сохраниться
Опечатка в длинном слове	Часть фрагментов остаётся	Сильное искажение меняет vector
Новая терминология	Representation по частям	Значение может быть неизвестно

Для русского текста сравнивают пользу character information с другими решениями на одних данных, а не считают её универсальным исправлением OOV.

Word2Vec, GloVe и fastText

Все три семейства относятся к static representations, если итоговый word vector не меняется от предложения к предложению.

Метод	Основной сигнал	Работа с новой записью
Word2Vec	Предсказание контекстов	Обычно lookup / OOV policy
GloVe	Global co-occurrence	Обычно lookup / OOV policy
fastText	Контексты и character n-grams	Возможен subword composition

Во всех случаях применение зависит от corpus и формата сохранённой модели. Static representation не различает отдельные значения слова по текущему контексту.

Static embeddings и polysemy

Один vector для разных употреблений

В static lookup слово bank получает одинаковый vector в предложениях о кредите и о берегу реки. Все наблюдавшиеся значения объединяются в одном representation. Если corpus содержит одно значение значительно чаще, оно может сильнее влиять на ближайших соседей. Это ограничение связано с единицей representation, а не только с размером vector.

Значение контекста для задачи

Для некоторых задач усреднённой информации о слове достаточно: контекст может обрабатывать следующая модель. Но для точного сравнения употреблений нужны representations, которые меняются вместе с окружением. Этот переход приводит к contextual embeddings. Перед ним полезно разобраться, как вообще получить один vector для целого документа.

Sentence и document representations

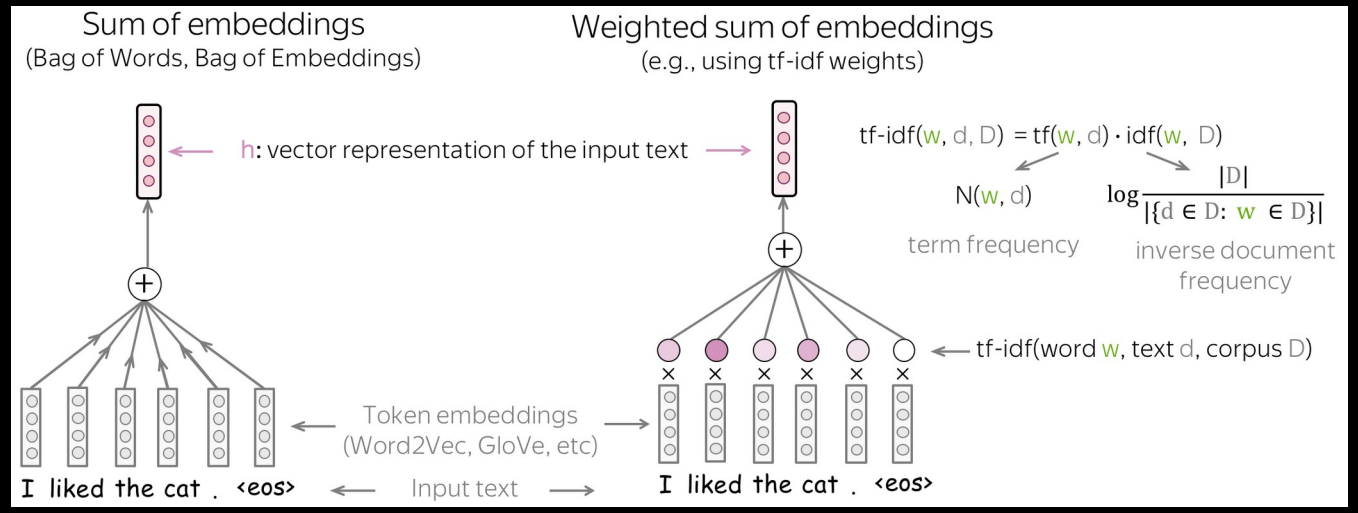
Один объект — один vector

При classification или retrieval часто требуется vector фиксированной длины для текста переменной длины. Он должен сохранять информацию, полезную для сравнения или предсказания class label. Это можно получить из lexical features, через объединение word vectors или с помощью модели, обученной представлять целые тексты.

Выбор единицы текста

Sentence, passage и document различаются объёмом и структурой. Один длинный документ может содержать несколько тем, поэтому единый vector способен смешивать разные смысловые части. Иногда полезнее представлять отдельные passages и хранить связь с исходным документом. Решение зависит от того, что именно должна возвращать система.

Bag of Embeddings



Bag of Embeddings объединяет word vectors, например усредняет их по тексту. Размер результата не зависит от количества слов. Weighted variant меняет вклад отдельных элементов; на схеме показаны оба подхода. Такое representation просто получить, но оно не учитывает порядок слов само по себе. Если два текста содержат одинаковый набор vectors с одинаковыми весами, результат будет одинаковым.

Weighted pooling

Вклад отдельных слов

При weighted pooling одни word vectors влияют на итоговое representation сильнее других. Веса могут задаваться статистикой corpus, например IDF, или обучаться для задачи. Это позволяет уменьшить вклад общего фона и выделить более характерные элементы. Правило выбора весов является частью метода и должно быть зафиксировано.

Ограничения объединения

Weighting не восстанавливает информацию, которую исходные vectors не содержат. Если static embedding объединяет несколько значений слова, одних весов может быть недостаточно для их разделения. Необходимо также определить обработку OOV, пустого текста и normalization результата. Все эти решения проверяют вместе, поскольку они влияют на similarity scores.

Потери при простом averaging

Пример показывает ограничение representation, которое использует одинаковые word vectors и не учитывает порядок.

Текст A	Текст B	Что различается
«Собака укусила человека»	«Человек укусил собаку»	Роли участников
«Товар хороший, сервис плохой»	«Товар плохой, сервис хороший»	Объект оценки

После приведения словоформ к общим единицам наборы элементов могут совпасть. Простое averaging не различит порядок и связи, если они не закодированы на предыдущем этапе.

TF-IDF для документов

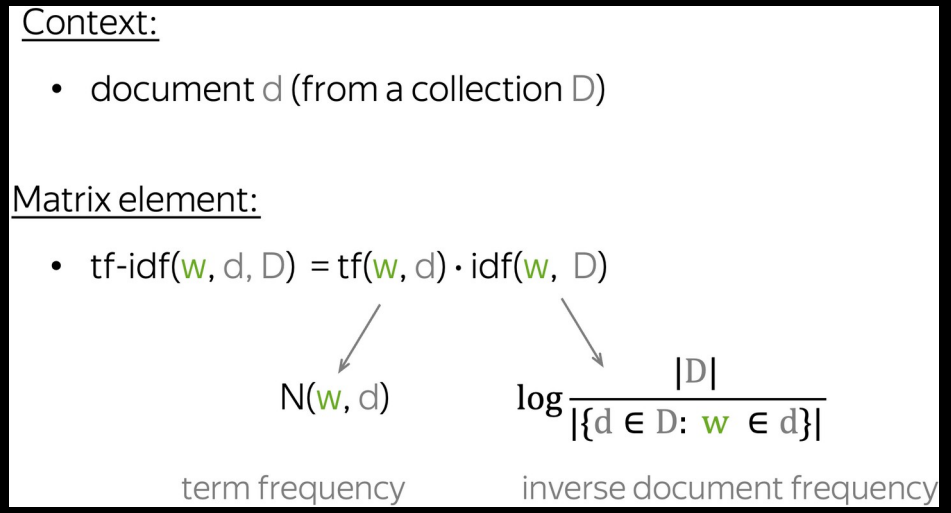
Lexical baseline

TF-IDF описывает документ через vocabulary corpus. Разные документы могут сравниваться по пересечению взвешенных terms или использоваться как вход classifier. Character и word n-grams расширяют набор наблюдаемых lexical patterns. В отличие от усреднения pretrained vectors, здесь представление непосредственно связано с выбранными единицами текста.

Сильные и слабые стороны

Подход сохраняет точные названия, идентификаторы и редкие сочетания, если они входят в vocabulary. Но одинаковый смысл без общих features может остаться незамеченным. Поэтому TF-IDF полезен и как самостоятельное решение, и как контроль при оценке embeddings. Сравнение должно учитывать качество, время обработки и объём хранения.

Latent Semantic Analysis



LSA строит компактное representation на основе term–document matrix. На схеме показаны TF-IDF weights, которыми можно заполнить исходную матрицу. Затем dimensionality reduction объединяет закономерности совместного употребления слов. Документы получают координаты в новом пространстве общих направлений. Важно различать исходные lexical features и результат этого преобразования; детали matrix factorization здесь не рассматриваются.

doc2vec / Paragraph Vector

Representation документа при обучении

doc2vec дополняет обучение слов представлением документа или paragraph. Document ID используется для выбора обучаемого vector, который помогает предсказывать слова. Тем самым модель учится сохранять информацию, полезную для описания данного текста. ID не является готовым смысловым признаком: значение получают параметры vector, связанные с этим ID.

Семейство методов

В оригинальной работе рассматриваются PV-DM и PV-DBOW. Они различаются тем, как document vector участвует в предсказании слов и используется ли локальный контекст. Название doc2vec обычно обозначает это семейство, а не один универсальный способ получения sentence embeddings. Отдельно необходимо определить обработку новых документов.

PV-DM

Distributed Memory model использует document vector вместе с контекстом слов. Таблица передаёт смысл компонентов без архитектурных деталей.

Компонент	Роль в предсказании
Document ID	Выбирает обучаемый document vector
Context words	Дают локальное окружение позиции
Target word	Задаёт обучающий ответ

Document vector помогает объяснять содержание, которое не полностью задано коротким локальным контекстом. Он обучается совместно с остальными параметрами.

PV-DBOW

Предсказание слов документа

Distributed Bag of Words model использует document vector для предсказания слов, встречающихся в документе. В этой постановке локальное окно соседних слов не является обязательным входом для получения данного сигнала. Представление обучается так, чтобы быть полезным для восстановления информации о содержании документа.

Что метод сохраняет

Document vector может отражать тему и другие регулярности текста, но не обязан сохранять все факты и порядок высказываний. PV-DBOW и PV-DM оценивают по downstream quality, а не по сходству названий с Word2Vec. Результат зависит от corpus, длины документов, training settings и процедуры inference для новых текстов.

doc2vec: inference нового документа

Новый ID не имеет обученного vector

В training corpus каждый document ID связан со своим vector. Для нового текста такого обученного элемента ещё нет. Классическая процедура inference фиксирует параметры модели и подбирает representation нового документа по его словам. Это отличается от простого lookup и от одного forward pass типичного sentence encoder.

Практические последствия

Число шагов inference и другие настройки могут влиять на скорость и устойчивость результата. Их нужно фиксировать при сравнении документов. Сходство representations, полученных разными способами, не следует принимать как самоочевидное. В Gensim полезно различать сохранённые vectors training documents и representations, вычисленные для новых текстов.

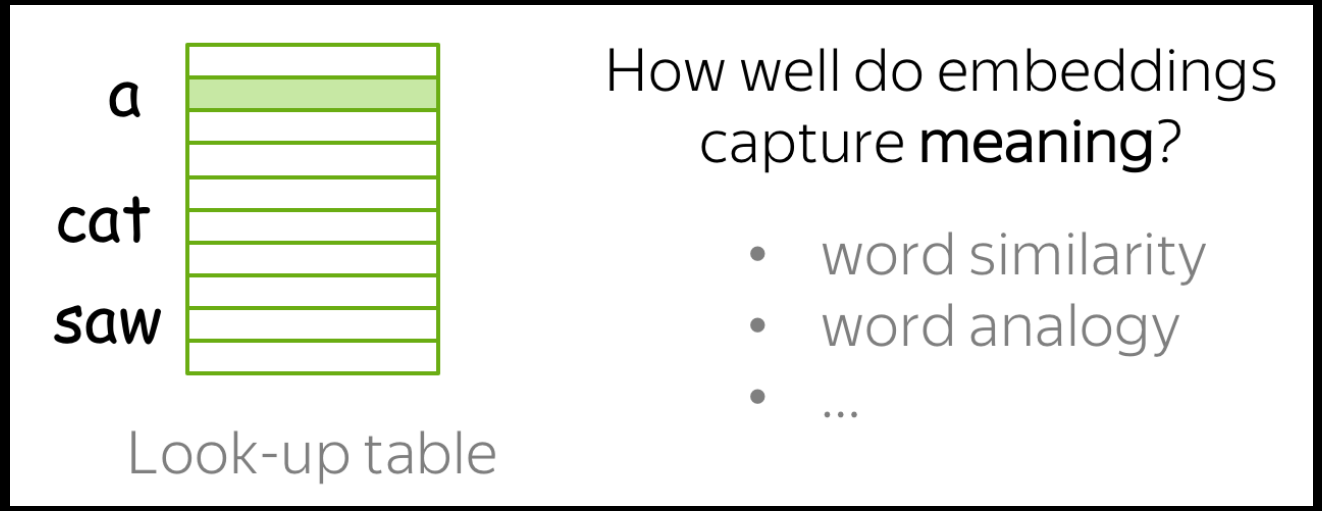
Document representations: сравнение

Подходы различаются источником признаков и тем, что требуется для обработки нового текста.

Подход	Representation	Новый текст
TF-IDF	Веса lexical features	Transform по train vocabulary
Word-vector pooling	Объединение word vectors	Lookup и pooling
doc2vec	Обученный document vector	Отдельная inference procedure
Sentence encoder	Vector целого текста	Forward pass модели

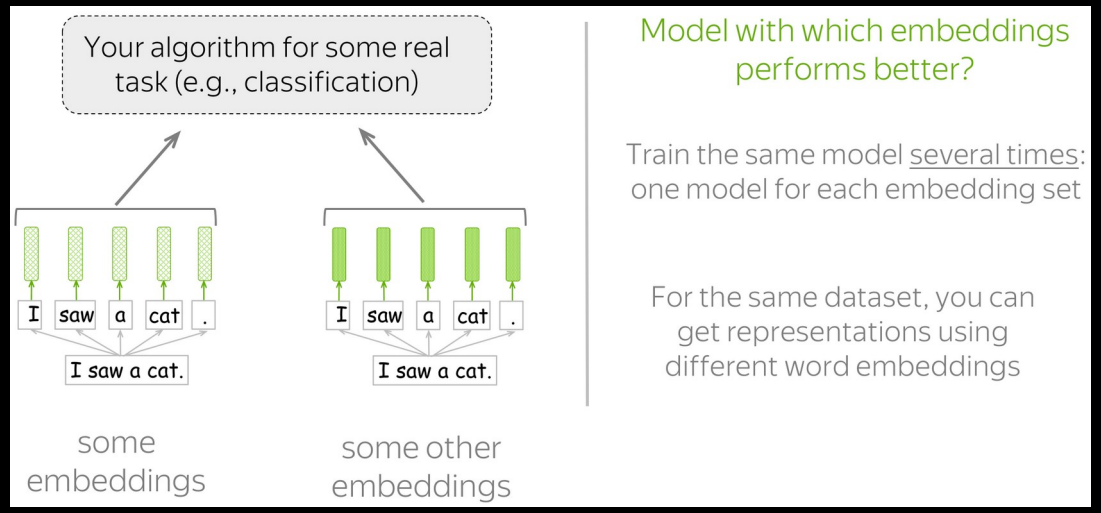
Сравнение проводят на одной задаче и с одинаковым доступом к данным. При оценке учитывают не только размер vector, но и качество и стоимость его получения.

Intrinsic evaluation



Intrinsic evaluation оценивает свойства representations через отдельную проверочную задачу: например, сходство слов или analogies. Она удобна для анализа пространства и сравнений на заранее определённом benchmark. Однако результат зависит от устройства benchmark и не всегда предсказывает качество конкретного приложения. Высокий score здесь не заменяет проверку retrieval или classification.

Extrinsic evaluation



Extrinsic evaluation проверяет, помогают ли representations решать downstream task. Например, embeddings используются как вход classifier, после чего оценивается качество его ответов. Сравнение требует одинаковых данных и согласованной процедуры обучения. Иначе изменение результата может объясняться другим classifier, дополнительным corpus или настройками, а не самими representations.

Evaluation representations

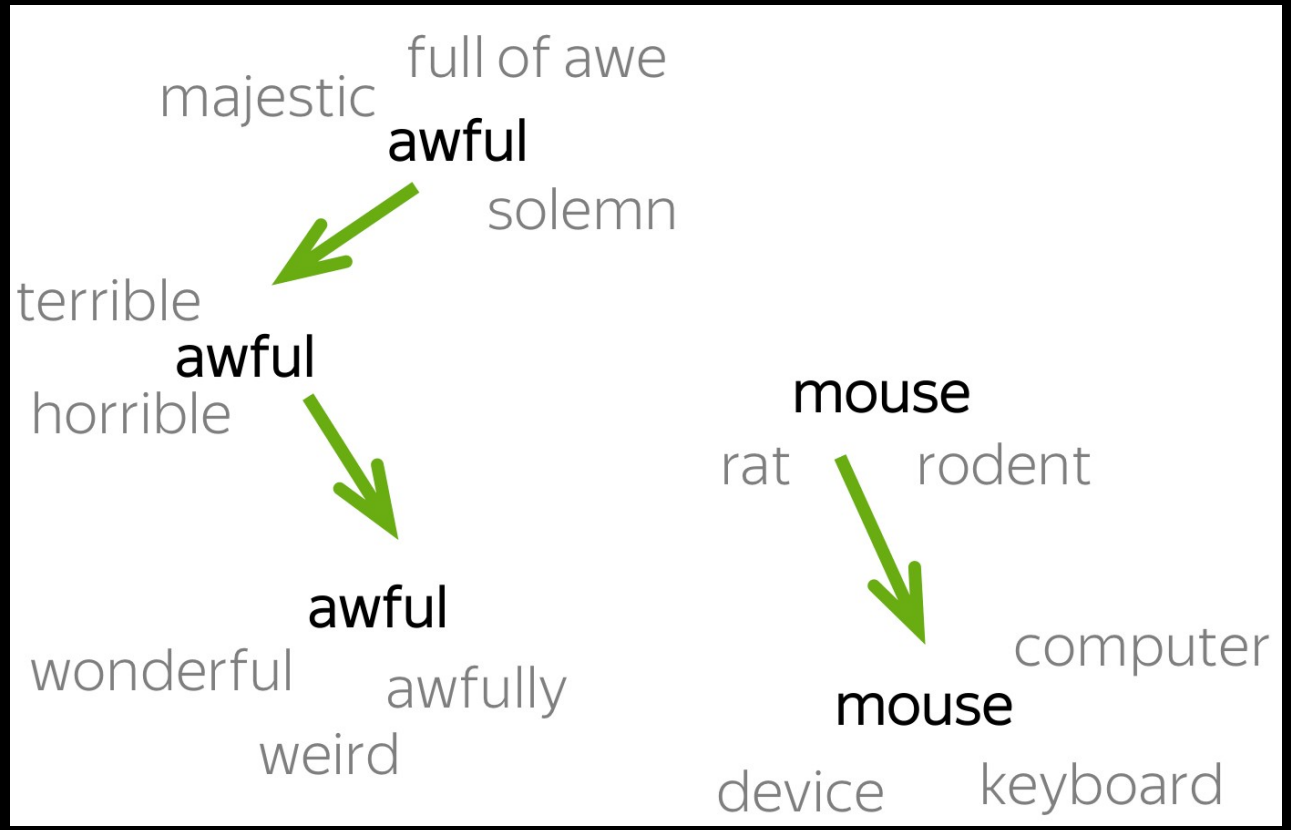
Сопоставимые условия

Фиксируют train/dev/test split, labels, preprocessing и критерии качества. При сравнении static embeddings проверяют coverage; при сравнении sentence encoders — truncation и способы encoding. Если меняются сразу representation, classifier и training data, отдельный эффект embeddings определить трудно. Полезно изменять компоненты последовательно.

Качество и затраты

Помимо итоговой metric учитывают время получения vectors, объём памяти и скорость поиска. На редких терминах, длинных документах и разных языках качество может различаться. Анализ по группам примеров дополняет средний score. Независимый test используют после выбора конфигурации; многочисленные попытки подбора по test искажают итоговую оценку.

Corpus и semantic change



Значения и контексты слов меняются со временем. На примере из курса показана идея анализа таких изменений через representations, обученные на разных периодах. Прямое сравнение координат требует согласования пространств; различие соседей также может зависеть от объёма и состава данных. Для прикладной системы это означает необходимость проверять embeddings на актуальном языке и терминологии.

Static и contextual embeddings

Ключевое различие — зависит ли representation конкретного token от окружающего текста при применении модели.

Свойство	Static	Contextual
Единица lookup	Слово из vocabulary	Употребление в последовательности
Одна запись в разных фразах	Один word vector	Разные hidden states
Примеры	Word2Vec, GloVe, fastText	ELMo, BERT

Contextual representation не гарантирует правильную интерпретацию каждого случая, но позволяет модели учитывать окружение непосредственно при вычислении vector.

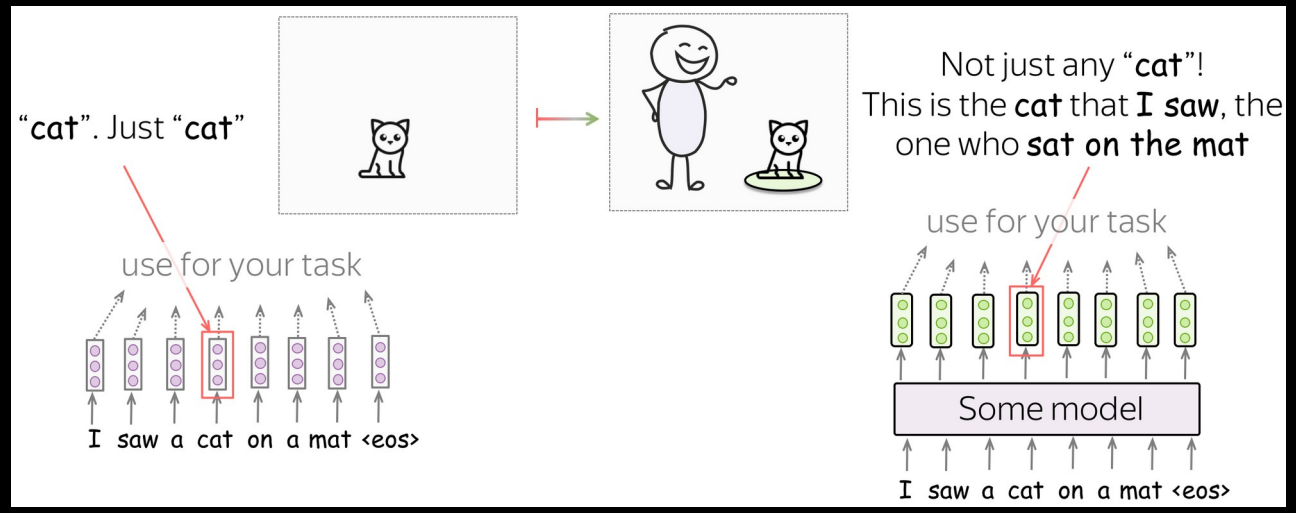
Context: пример слова bank

Одинаковая запись употребляется в разных контекстах. Пример иллюстрирует задачу различения значений, а не измеренный результат конкретной модели.

Контекст	Употребление bank	Что требуется от representation
The bank approved a loan.	Финансовая организация	Учесть approved и loan
They sat on the river bank.	Берег реки	Учесть river и sat

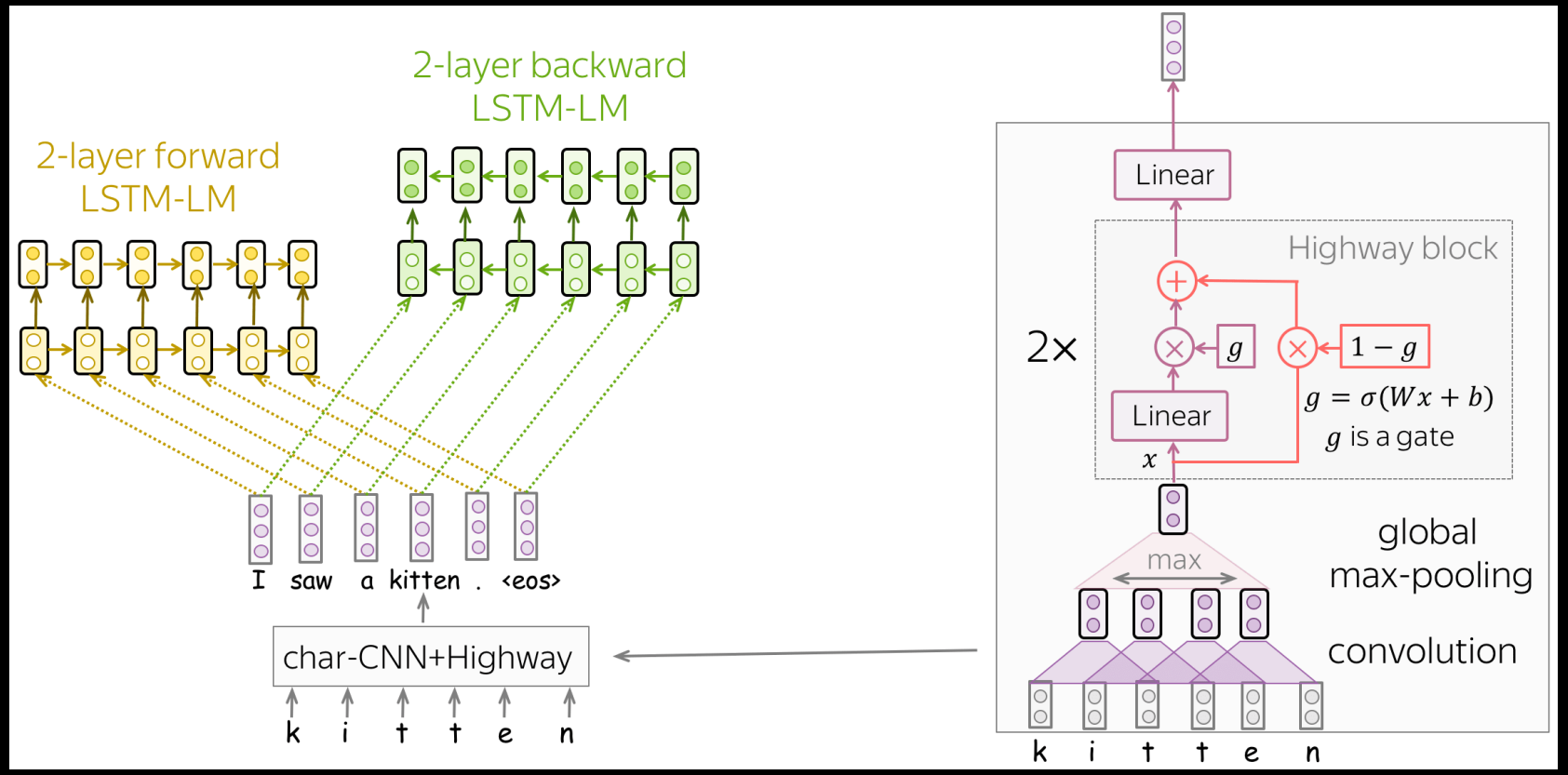
Static lookup возвращает одинаковый word vector. Contextual encoder может вычислить разные representations для этих двух вхождений.

Words-in-context



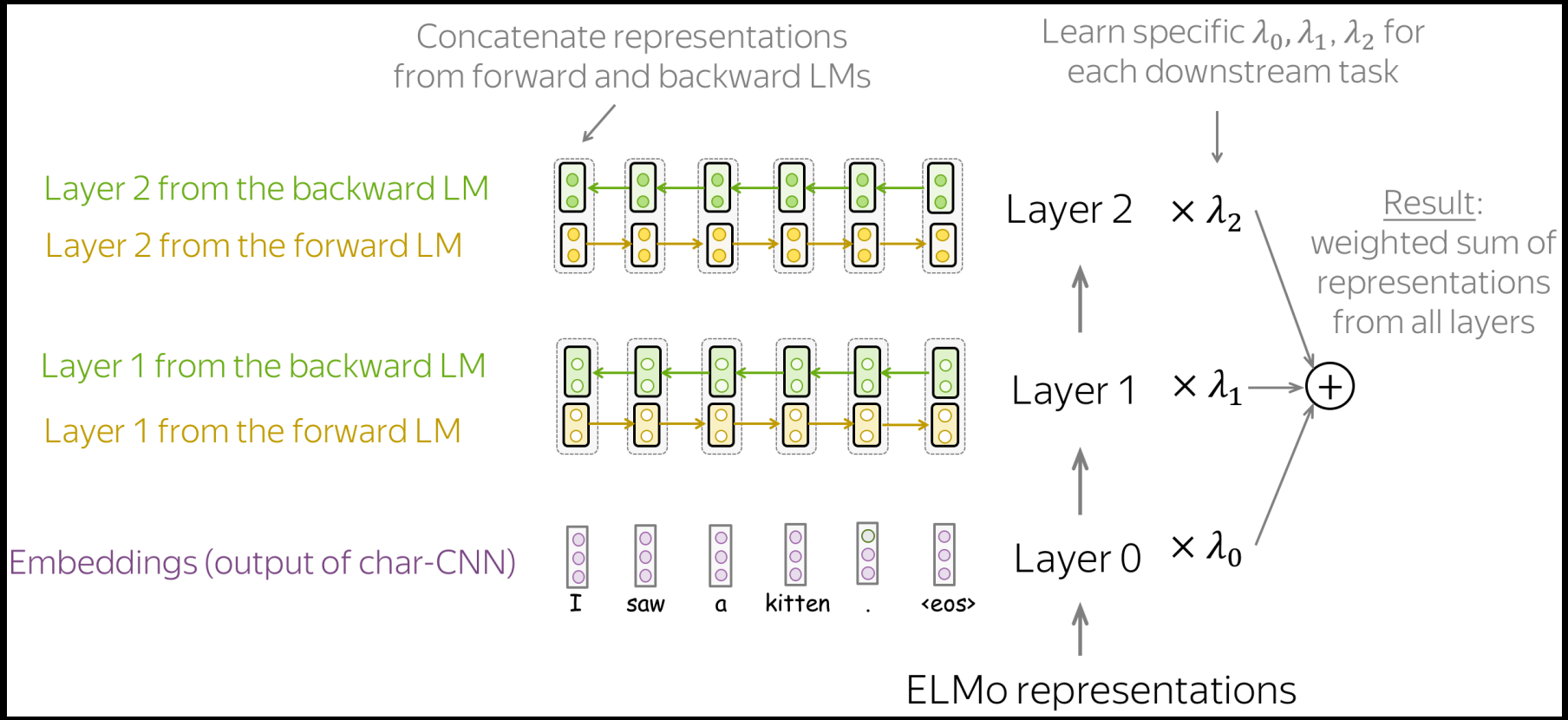
Contextual model получает всю доступную последовательность и строит representation для каждой её позиции. Входные token embeddings ещё могут быть одинаковыми для одной записи, но последующие hidden states учитывают окружение. На схеме курса этот переход отделён от простого word lookup. В результате объектом анализа становится употребление слова, а не только его тип в vocabulary.

ELMo



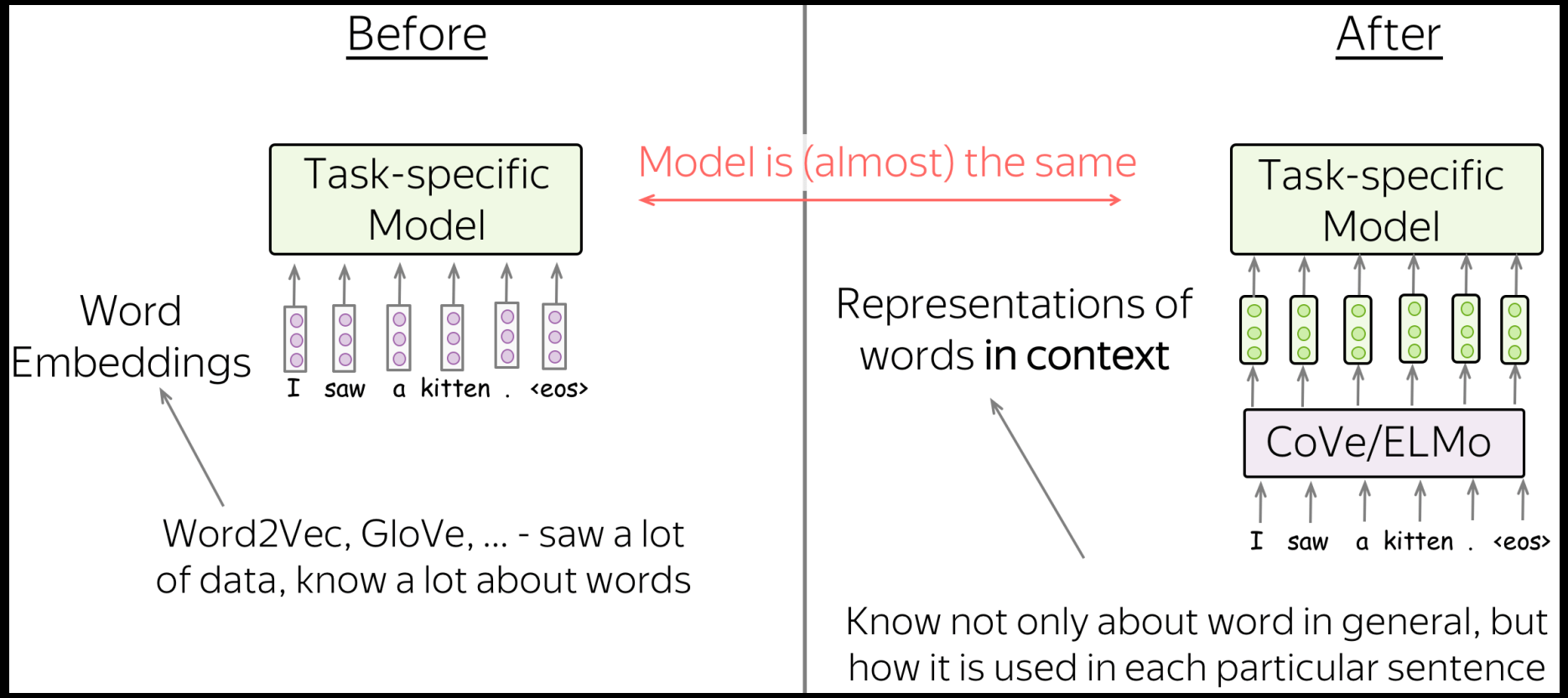
ELMo — Embeddings from Language Models. Forward и backward LM дают representations с учётом разных сторон контекста.

ELMo: representations разных слоёв



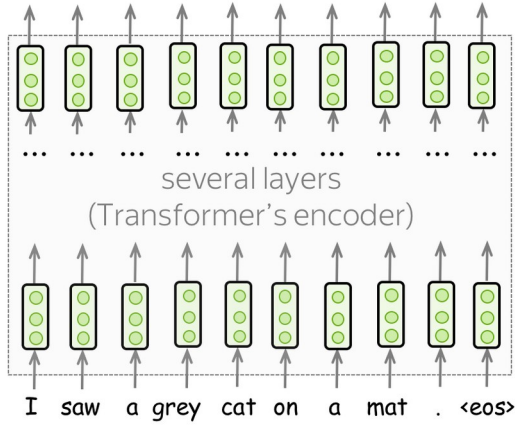
ELMo объединяет representations нескольких слоёв. Веса их сочетания можно обучать для downstream task.

ELMo в downstream task



Contextual representations передаются модели downstream task. Переносится способ описания текста, а labels задаются новой задачей.

BERT



Model architecture:

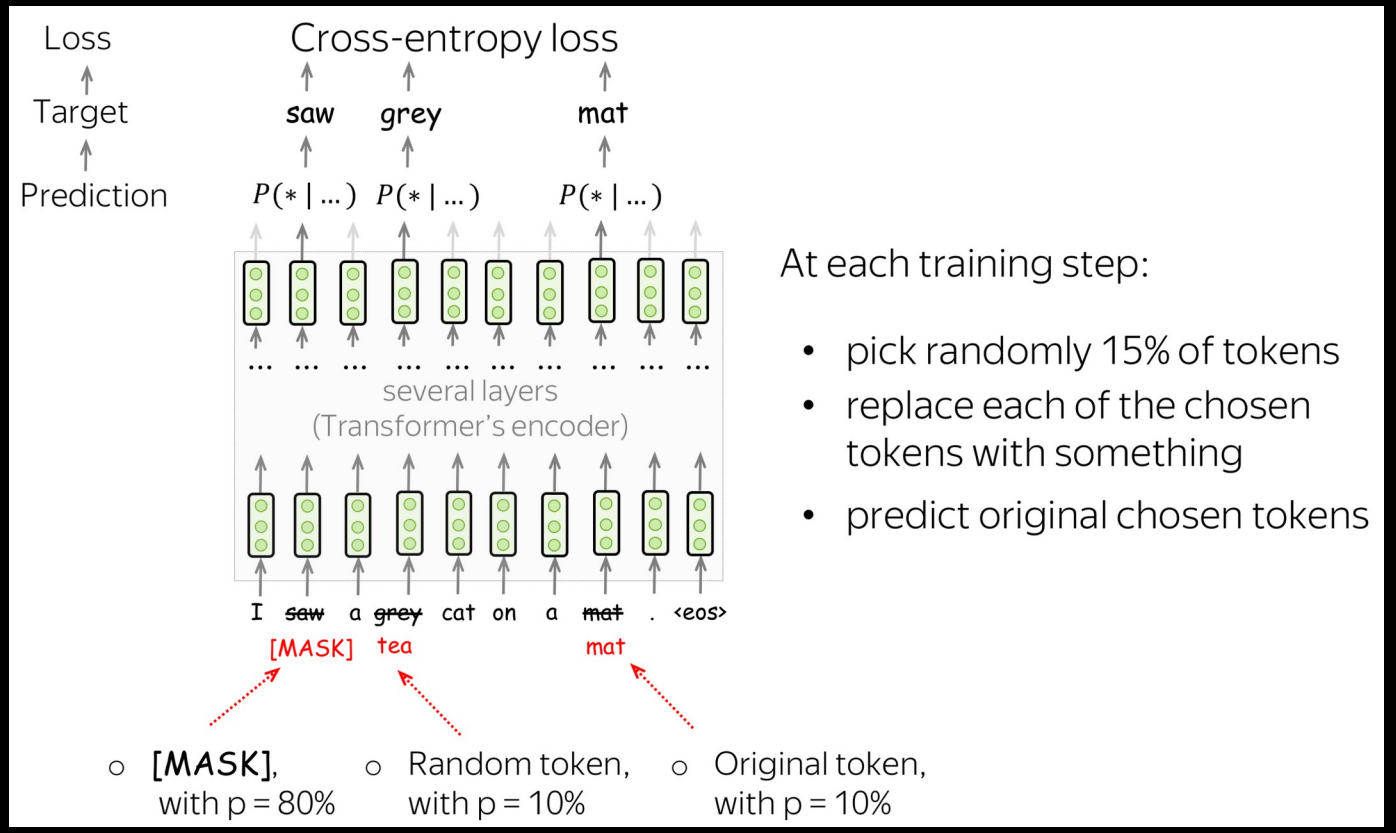
- Transformer's encoder

What is special about it:

- Training objectives
  - MLM: Masked language modeling
  - NSP: Next sentence prediction
- The way it is used
  - No task-specific models

BERT использует Transformer encoder для вычисления representations с учётом контекста с обеих сторон. Одна pretrained model затем может адаптироваться к разным downstream tasks. Это отличается от идеи переносить только таблицу static word embeddings. В этом блоке достаточно понимать данный принцип; устройство attention и архитектурные детали рассматриваются далее в курсе.

Masked Language Modeling

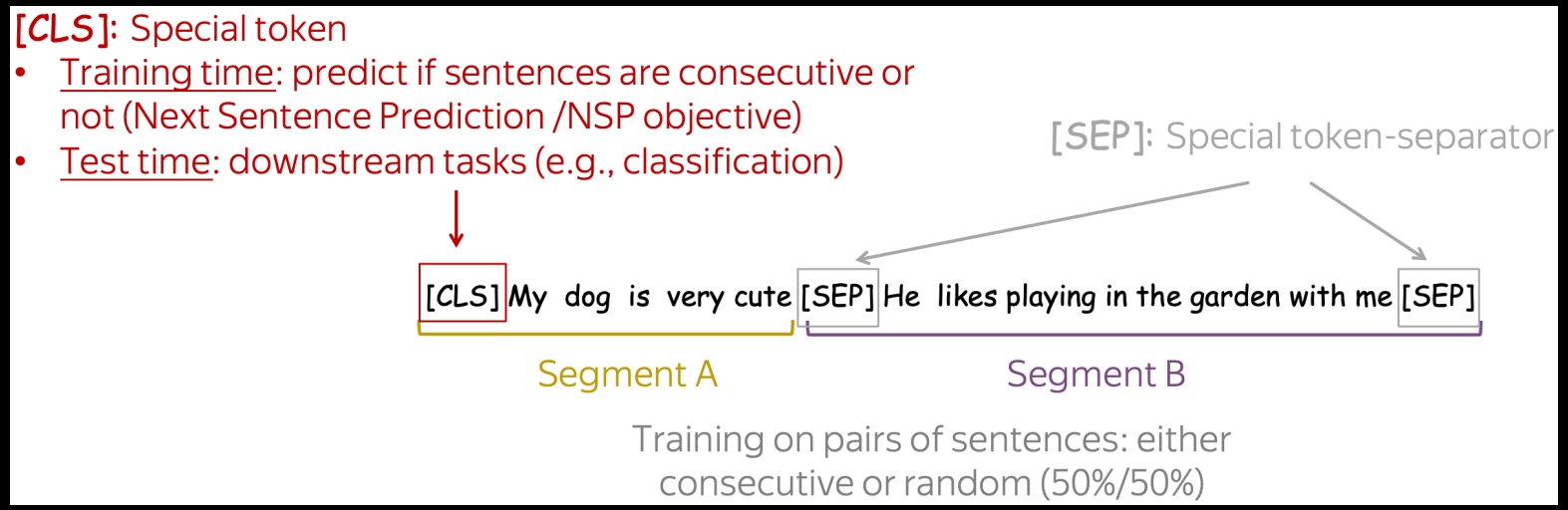


At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something
- predict original chosen tokens

MLM восстанавливает выбранные tokens по контексту. На схеме — процедура pretraining оригинального BERT.

Tokenizer и вход BERT



Tokenizer переводит текст в token IDs и добавляет специальные элементы, предусмотренные моделью. В оригинальном BERT используются WordPiece и tokens вроде CLS и SEP. Одно слово может занимать несколько позиций. Поэтому vector отдельного token не всегда является representation целого слова. Tokenizer, vocabulary и pretrained weights должны соответствовать друг другу.

Hidden states и pooling

Token-level выход

Encoder возвращает hidden states для позиций входной последовательности. Они отражают текущий контекст и могут использоваться в sequence labeling или других задачах на уровне tokens. Длина последовательности и embedding dimension — разные оси результата. При работе со словами, разбитыми на несколько subwords, требуется определить правило объединения позиций.

Sentence-level выход

Чтобы получить один vector для текста, используют pooling или специально обученный output. Возможны mean pooling, CLS representation и другие варианты. Выбор должен соответствовать training recipe модели. Нельзя считать любой vector первого token универсальным representation предложения: полезность определяется обучением и evaluation на нужной задаче.

BERT и sentence embeddings

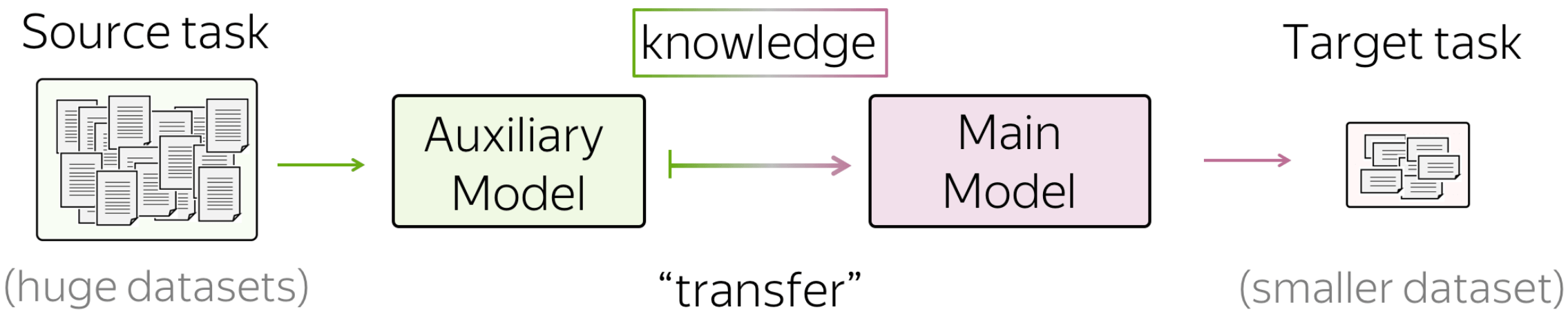
Pretraining не задаёт готовую similarity metric

BERT обучается прежде всего строить representations, полезные для pretraining objectives и последующей адаптации. Это не означает, что cosine similarity между любыми pooled outputs сразу хорошо измеряет близость предложений. Устройство пространства зависит от обучения; representation для token prediction и representation для semantic retrieval требуют разных проверок.

Специализированный sentence encoder

Sentence embedding model дополнительно обучают сопоставлять целые тексты. Обычно для этого используются пары, relevance labels, similarity data или contrastive objectives. Поэтому сравнение «TF-IDF против BERT» слишком неопределённо: необходимо указать checkpoint, pooling, normalization, способ адаптации и downstream evaluation.

Pretraining и downstream adaptation



Transfer learning использует параметры, полученные на source task или corpus, при решении target task. Pretraining формирует общие representations; дальнейшее обучение может адаптировать их к новым labels или домену. На схеме показан перенос знаний между задачами. Польза такого переноса зависит от совместимости данных и того, какую информацию сохранила pretrained model.

Feature extraction и fine-tuning

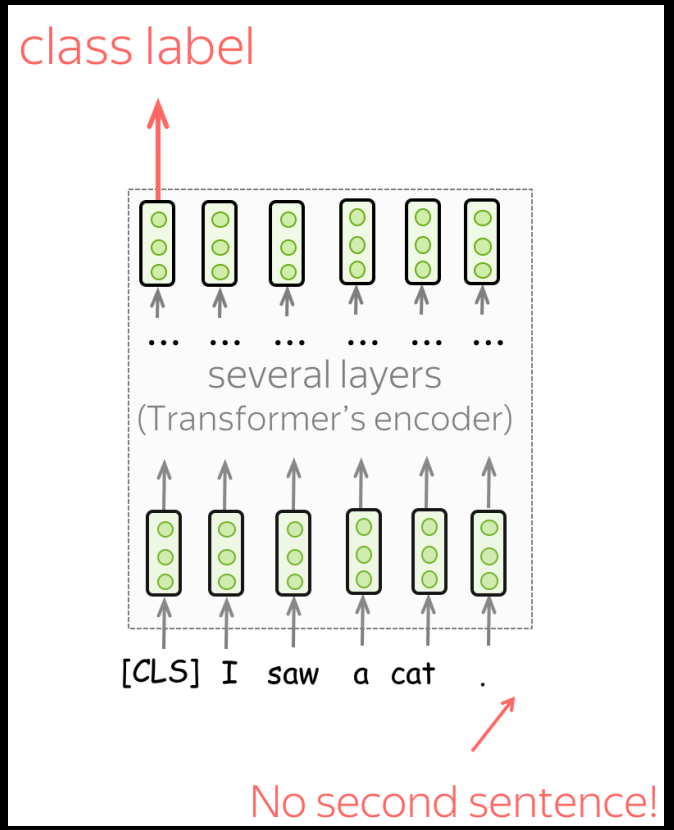
Frozen encoder

При feature extraction pretrained encoder фиксируют и используют для вычисления representations. Затем обучают небольшой classifier или применяют similarity search. Это упрощает эксперимент и позволяет заранее сохранить vectors. Однако encoder не адаптируется к особенностям целевой задачи; качество ограничено исходным representation и выбранным pooling.

Fine-tuning

При fine-tuning часть или все параметры модели изменяются по целевым данным. Это может улучшить качество, но требует корректного split и осторожного подбора training settings. После изменения encoder ранее сохранённые vectors обычно нужно пересчитать. Сравнение frozen и fine-tuned вариантов должно учитывать стоимость обучения и риск overfitting.

Backbone и task head



Pretrained backbone строит contextual representations. Task head преобразует их в ответ конкретной задачи, например class probabilities. На схеме курса показан вариант BERT для sentence classification. При переносе между задачами backbone можно использовать повторно, а head выбирается под новый формат ответа. Его выход не следует путать с embedding: score класса и representation текста выполняют разные функции.

Domain adaptation

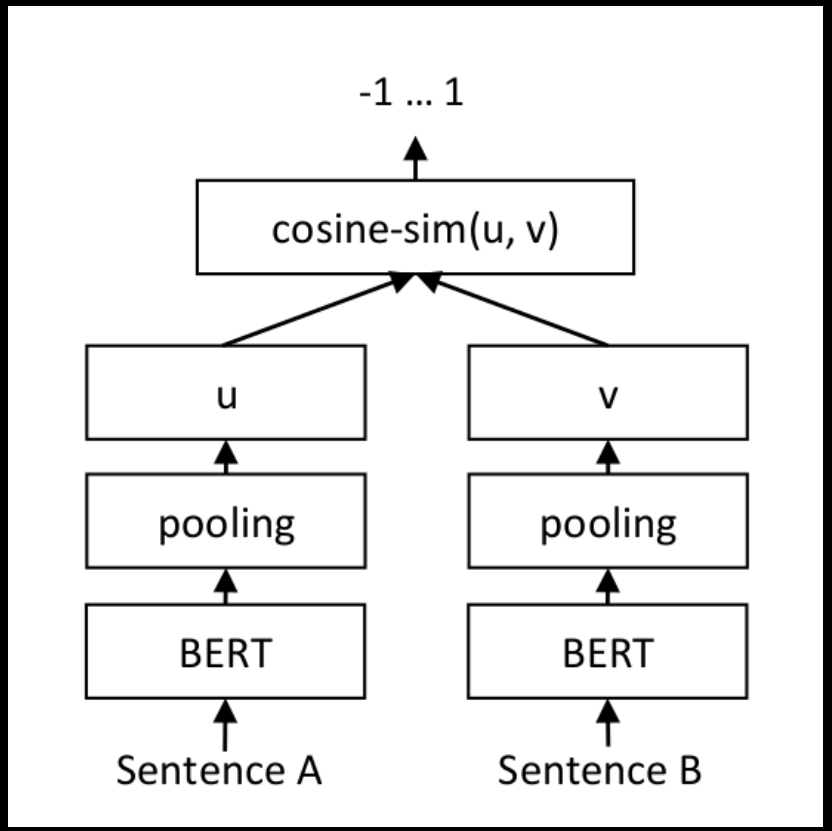
Несоответствие данных

Model может быть обучена на новостях или Wikipedia, а применяться к юридическим документам, переписке или описаниям товаров. Меняются vocabulary, жанр и распределение смыслов. Поэтому высокий benchmark score на общей задаче не гарантирует подходящего representation в новом домене. Важны реальные тексты и характерные поисковые запросы.

Выбор адаптации

Сначала проверяют pretrained baseline и анализируют ошибки. Затем рассматривают дополнительное обучение на доменных текстах или на размеченных парах нужной задачи. Эти варианты используют разный сигнал и могут давать разные результаты. Evaluation должна показывать, улучшилось ли целевое поведение, а не только насколько модель приспособилась к training corpus.

Sentence-BERT и Siamese architecture



Sentence-BERT обучает representations целых предложений с помощью общей encoder architecture для разных текстов. Shared weights означают, что сравниваемые тексты обрабатываются согласованным способом. Полученные vectors можно сохранять и повторно использовать. Это отличается от обработки каждой пары совместной моделью, где representation одного текста зависит от конкретного второго текста.

Источник: Sentence Transformers

Обучающие пары для sentence embeddings

Учебные примеры показывают требуемое отношение между текстами. Конкретная loss function может использовать пары, labels или группы candidates.

Текст / query	Связанный текст	Обучающий сигнал
«Как вернуть заказ?»	«Инструкция по возврату товара»	Positive pair
«Как вернуть заказ?»	«Условия рассрочки»	Negative pair
«Отмена подписки»	«Как отключить автопродление»	Positive pair

Negatives должны отражать реальные сложности задачи. Слишком простые negatives не учат различать близкие по теме, но нерелевантные ответы.

Pooling для sentence representation

Pooling превращает набор token representations в один vector. Выбранная операция должна соответствовать обучению модели.

Способ	Принцип	Условие применения
Mean pooling	Усреднение нужных позиций	Padding исключается по mask
CLS pooling	Использование специальной позиции	Модель обучена использовать её
Learned pooling	Обучаемое объединение	Нужны соответствующие weights

Произвольная смена pooling может ухудшить embeddings. Одинаковая размерность результата не делает разные способы взаимозаменяемыми.

Encoding нового текста

Согласованные этапы

Encoder получает tokens, attention mask и другие входы, предусмотренные моделью. Затем вычисляются hidden states, pooling и, если необходимо, normalization. В типичном sentence encoder один forward pass возвращает vector фиксированной длины. Внешний API может скрывать эти этапы, но их настройки продолжают влиять на результат.

Что фиксируется вместе с vector

Для воспроизводимости сохраняют model identifier, revision, tokenizer, pooling, max length и normalization. При asymmetric retrieval дополнительно фиксируют query/document mode или prompt. Batch size обычно относится к организации вычислений, но truncation меняет сам доступный текст. Векторы без сведений об их происхождении трудно безопасно переиспользовать.

Similarity score и решение

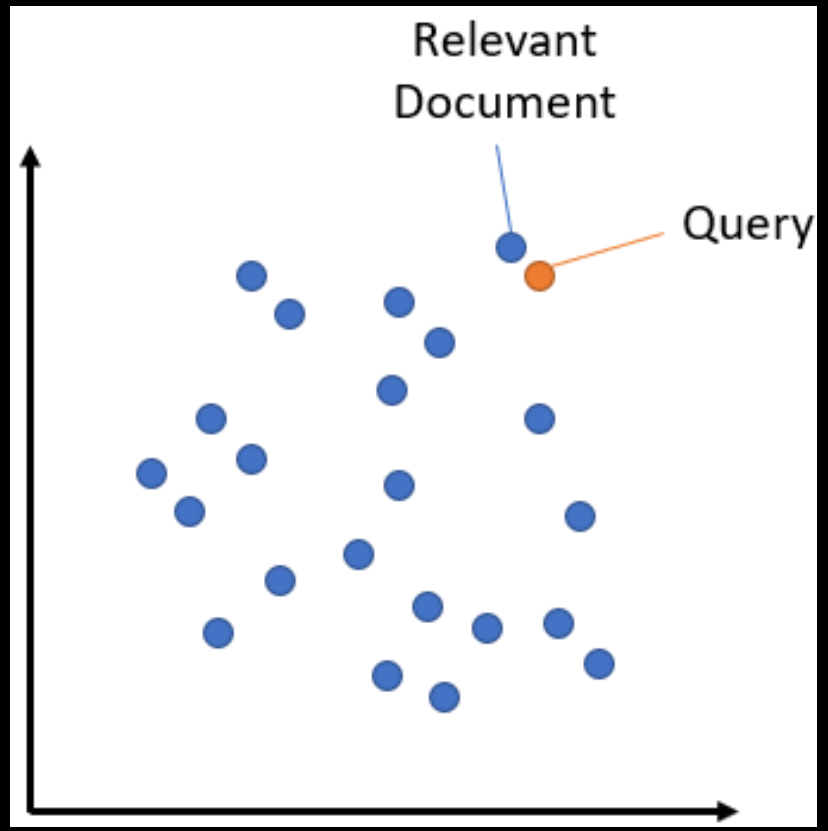
Score зависит от пространства

Similarity function сравнивает vectors, но её значение не является универсальной шкалой смысла. Одни модели используют cosine similarity, другие обучаются для dot product или иной меры. Normalization может менять связь между этими способами сравнения. Поэтому similarity function выбирается по документации конкретного checkpoint и проверяется на данных задачи.

Threshold и ranking

Ranking выбирает наиболее подходящие candidates относительно друг друга. Threshold дополнительно решает, достаточно ли высок score для принятия результата. Порог нельзя переносить между моделями без проверки. Для случаев без релевантного документа нужны отдельные примеры: наличие nearest neighbor не означает, что система нашла правильный ответ.

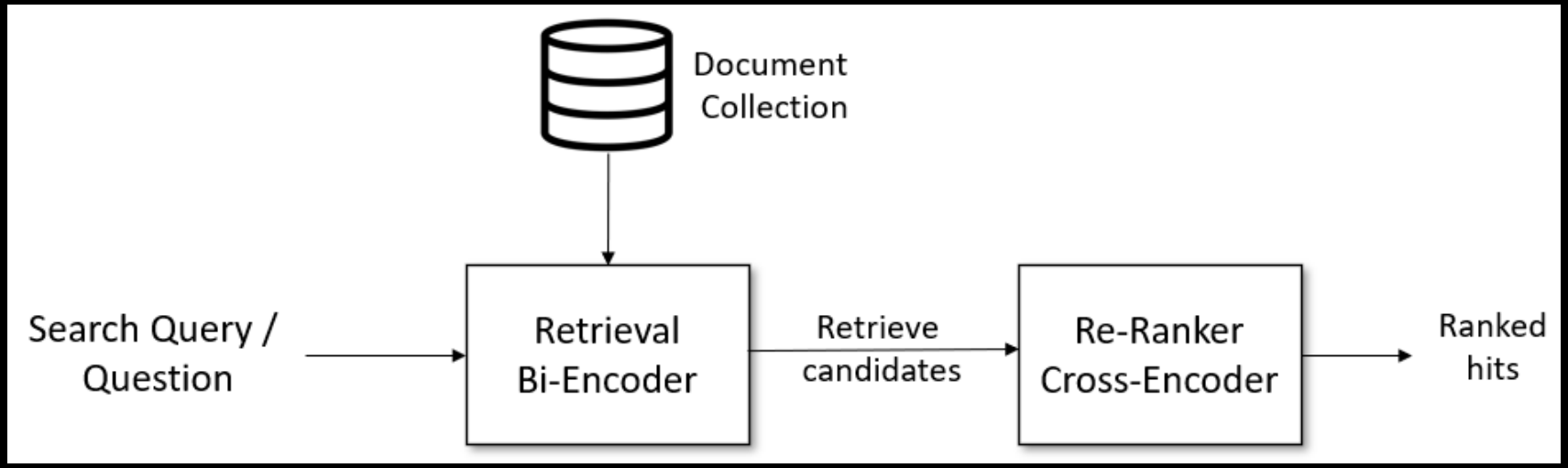
Bi-encoder retrieval



Bi-encoder вычисляет representation query и representation document раздельно. Document vectors можно подготовить заранее и хранить в index. Для нового query вычисляется один vector, который сравнивается с candidates. На схеме query и documents находятся в общем embedding space. Независимый encoding делает retrieval эффективным, но ограничивает взаимодействие между query и document сравнением готовых vectors.

Источник: Sentence Transformers

Retrieval pipeline



В большом corpus сначала получают небольшое множество candidates, затем при необходимости уточняют их порядок. На схеме Sentence Transformers retrieval и re-ranking представлены как разные этапы. Embedding model влияет на то, какие документы попадут в candidates. Если нужного документа нет на первом этапе, последующий re-ranker не сможет вернуть его без дополнительного поиска.

Источник: Sentence Transformers

Symmetric и asymmetric search

Разные роли текстов

При symmetric search сравниваются похожие по роли и длине объекты: например, две формулировки вопроса. При asymmetric search короткий query сопоставляется с более длинным passage, содержащим ответ. Требуемое отношение между ними не сводится к одинаковой формулировке: хороший ответ может содержать другие слова и иметь другую структуру.

Query и document encoding

Некоторые модели используют разные instructions или специальные modes для query и document. Поэтому один и тот же универсальный вызов без нужного prompt может не соответствовать training recipe. Выбор encoder и способ encoding проверяют на характерных парах. Для multilingual search дополнительно оценивают сочетания языков query и документов.

Chunking документов

Единица retrieval

Длинный документ часто разделяют на passages или chunks. Слишком крупный фрагмент может смешивать несколько тем, слишком маленький — терять контекст. Границы желательно согласовывать со структурой: заголовками, абзацами, предложениями. Размер chunk определяется доступным контекстом модели и тем, какой объём материала нужен для ответа.

Связь с источником

Для каждого chunk сохраняют document ID, положение в документе и полезные metadata. Это позволяет показать источник и восстановить соседние фрагменты. Overlap может помочь на границах, но создаёт близкие candidates и увеличивает объём index. Параметры chunking оценивают вместе с retrieval quality, а не выбирают только по числу tokens.

Vector index и ANN

Поиск по representations

При exact search query сравнивается со всеми candidate vectors. Для больших коллекций это может быть дорого. Approximate Nearest Neighbor search использует index, чтобы быстрее находить близкие элементы с возможной потерей части точных соседей. Скорость, память и полнота поиска образуют отдельный инженерный компромисс.

Разделение источников ошибки

Плохой результат может быть вызван encoder, chunking, similarity function или approximation в index. Эти причины нужно различать. Сначала полезно проверить качество representations на небольшой коллекции с exact search, затем оценить потери при переходе к ANN. Обновление encoder обычно требует пересчёта corpus vectors и перестройки соответствующего index.

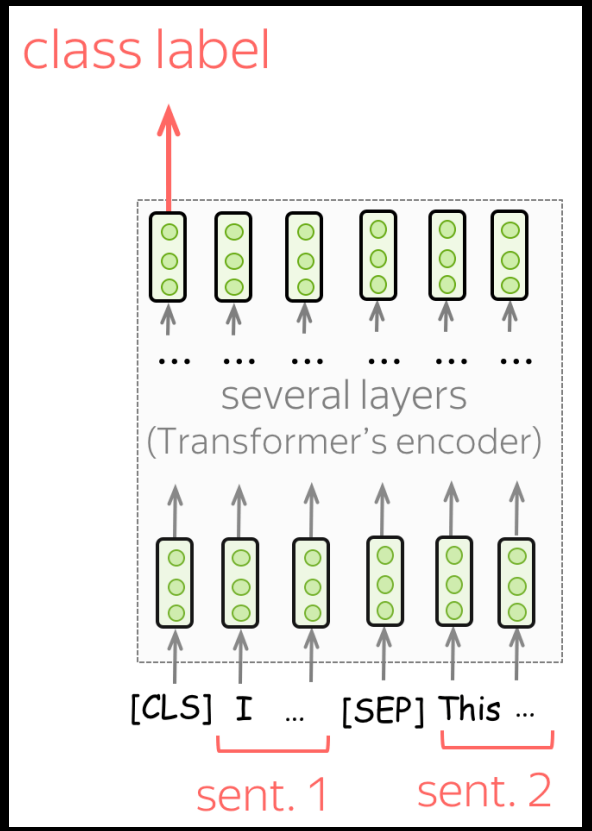
Lexical, dense и hybrid retrieval

Разные способы retrieval выделяют разные связи между query и документом. Hybrid approach сочетает несколько сигналов.

Подход	Что хорошо сохраняет	Типичный риск
Lexical	Точные terms и identifiers	Нет совпадающих формулировок
Dense	Связи между разными выражениями	Потеря точных деталей
Hybrid	Несколько видов evidence	Нужна настройка объединения

TF-IDF или BM25 могут быть полезны рядом с embeddings. Выбор и объединение candidates проверяют по relevance labels и эксплуатационным ограничениям.

Cross-encoder и re-ranking



Cross-encoder получает query и document совместно и оценивает их связь с учётом взаимодействия между tokens. На схеме показана обработка пары текстов одним BERT; task head можно обучить предсказывать relevance. Выход используется как score пары, а не как независимо сохраняемый document embedding. Такая обработка дороже для большого числа пар, поэтому модель часто применяется к небольшому набору candidates после retrieval.

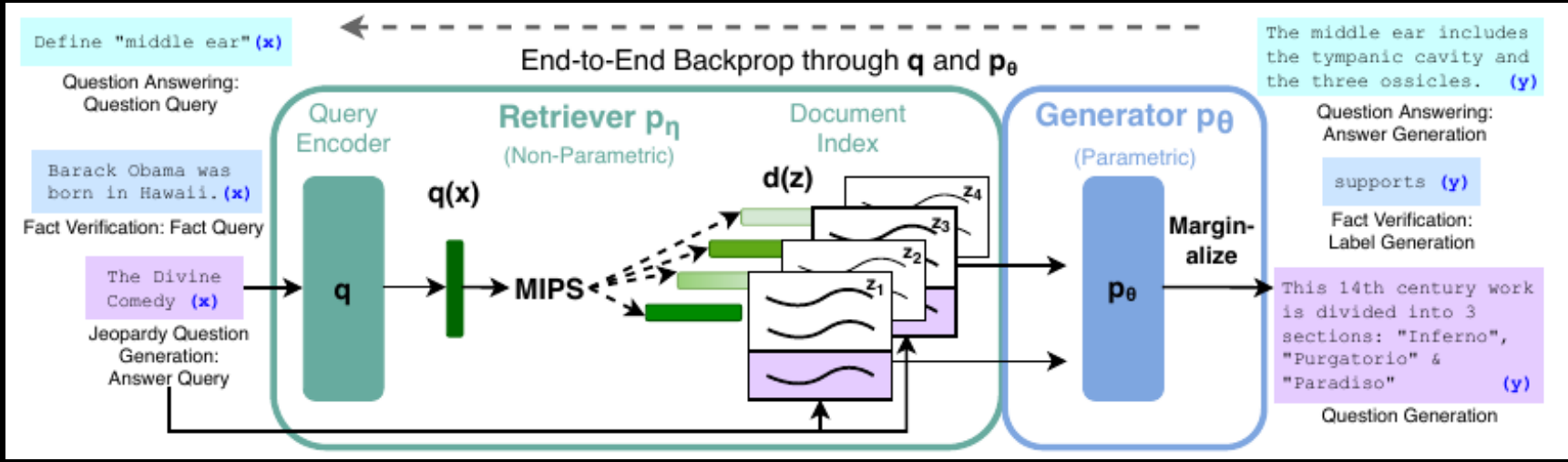
Retrieval evaluation

Учебный пример: для query существуют два релевантных документа, A и B. Система вернула следующий top-3.

Rank	Document	Relevance
1	C	Нет
2	A	Да
3	D	Нет

Recall@3 = 1/2: найден один из двух релевантных документов. Reciprocal Rank = 1/2: первый релевантный ответ стоит на втором месте. MRR — среднее Reciprocal Rank по всем queries.

Embeddings в RAG



В оригинальной схеме RAG retriever находит документы по representation query, а generator использует найденное содержание для ответа. Схема Lewis et al. показывает конкретную исследовательскую модель; современные системы могут организовывать обучение иначе. Общая роль embeddings остаётся прикладной: помочь получить релевантные внешние материалы. Сам vector не является ответом и не содержит проверяемой ссылки на факт.

Источник: Lewis et al., 2020 · Figure 1

Retrieval и grounded generation

Качество evidence

Даже корректно работающий generator ограничен найденными материалами. Если retriever вернул устаревший или нерелевантный passage, ответ может оказаться неверным. Поэтому качество RAG оценивают по этапам: были ли найдены нужные документы, поддерживают ли они ответ и корректно ли использованы при generation. Один общий показатель может скрывать причину ошибки.

Источники и обновление данных

Система должна сохранять связь chunk с исходным документом и учитывать актуальность и доступность материалов. Обновление corpus не равно обновлению encoder: это разные операции. Embeddings помогают выбирать содержание, но не гарантируют его истинность. Для ответа без достаточного evidence требуется отдельная логика, а не обязательный выбор ближайшего passage.

Сравнение на семинаре

TF-IDF baseline

Первое решение строится из TF-IDF и Logistic Regression на подготовленных текстах. Vocabulary и IDF оцениваются только по train. Затем фиксируются evaluation metrics и конкретные ошибки. Такой baseline позволяет проверить данные и получить понятную отправную точку без обучения большой encoder model.

Sentence embeddings

Во втором варианте те же тексты получают representations через pretrained Sentence Transformers. Classifier и split по возможности сохраняют сопоставимыми. Сравнивают quality, время encoding и поведение на разных типах примеров. Код и запуск относятся к семинару; лекция задаёт смысл компонентов и правила интерпретации результата.

Библиотеки и компоненты

Инструменты выбирают по операции. Наличие готового API не отменяет проверки данных, формата representation и настроек модели.

Инструмент	Назначение
scikit-learn	TF-IDF, classifiers, metrics, Pipeline
Gensim	Word vectors, Word2Vec, FastText, doc2vec
Hugging Face Transformers	Tokenizer, pretrained encoder, hidden states
Sentence Transformers	Text embeddings, similarity, retrieval

Для поиска также нужен способ хранения vectors и index. Модель, tokenizer, pooling и версия данных фиксируются как части одного решения.

Выбор pretrained model

Model card и назначение

Проверяют supported languages, training objective и тип рекомендуемой задачи. Модель для коротких paraphrases может не подходить для поиска длинных passages. Важны license, model revision, max sequence length, embedding dimension и ожидаемый query/document format. Размер checkpoint или популярность не заменяют evaluation на собственных примерах.

Ограничения применения

Truncation удаляет часть длинного текста; normalization и prompts меняют representations. Для новых языков и доменов качество может существенно отличаться от заявленных benchmark results. Выбор подтверждают небольшим размеченным набором, затем проверяют скорость и память. При смене модели пересматривают thresholds и совместимость существующего index.

Результаты лекции

Основные различия

Static embeddings связывают слово с одним vector; contextual embeddings описывают его употребление. Sentence и document representations требуют определённого способа объединения информации или отдельного обучения. Pretraining и fine-tuning используют разные данные и цели. В retrieval качество зависит одновременно от encoder, corpus, chunking и способа поиска.

Контрольные вопросы

Почему одинаковая dimension не означает совместимость пространств? Чем fastText OOV отличается от обычного lookup? Как получают representation нового документа в doc2vec? Почему pooled BERT не всегда является хорошим sentence encoder? Когда нужны разные query/document modes? Какие ошибки RAG невозможно исправить только заменой generator?

Материалы и источники

Elena Voita · NLP Course For You

Word Embeddings и Transfer Learning.

Word2Vec · GloVe · fastText

Mikolov et al.; Pennington et al.; Bojanowski et al.

Paragraph Vector / doc2vec

Le & Mikolov, 2014.

ELMo · BERT

Peters et al., 2018; Devlin et al., 2019.

Sentence-BERT

Reimers & Gurevych, 2019; Sentence Transformers.

Retrieval-Augmented Generation

Lewis et al., 2020.

Содержание занятия: программа блока «Основы NLP и векторные представления», 24 октября. Практическая реализация — на семинаре.